

15.12.2025

Динамическое программирование. Жадные алгоритмы.

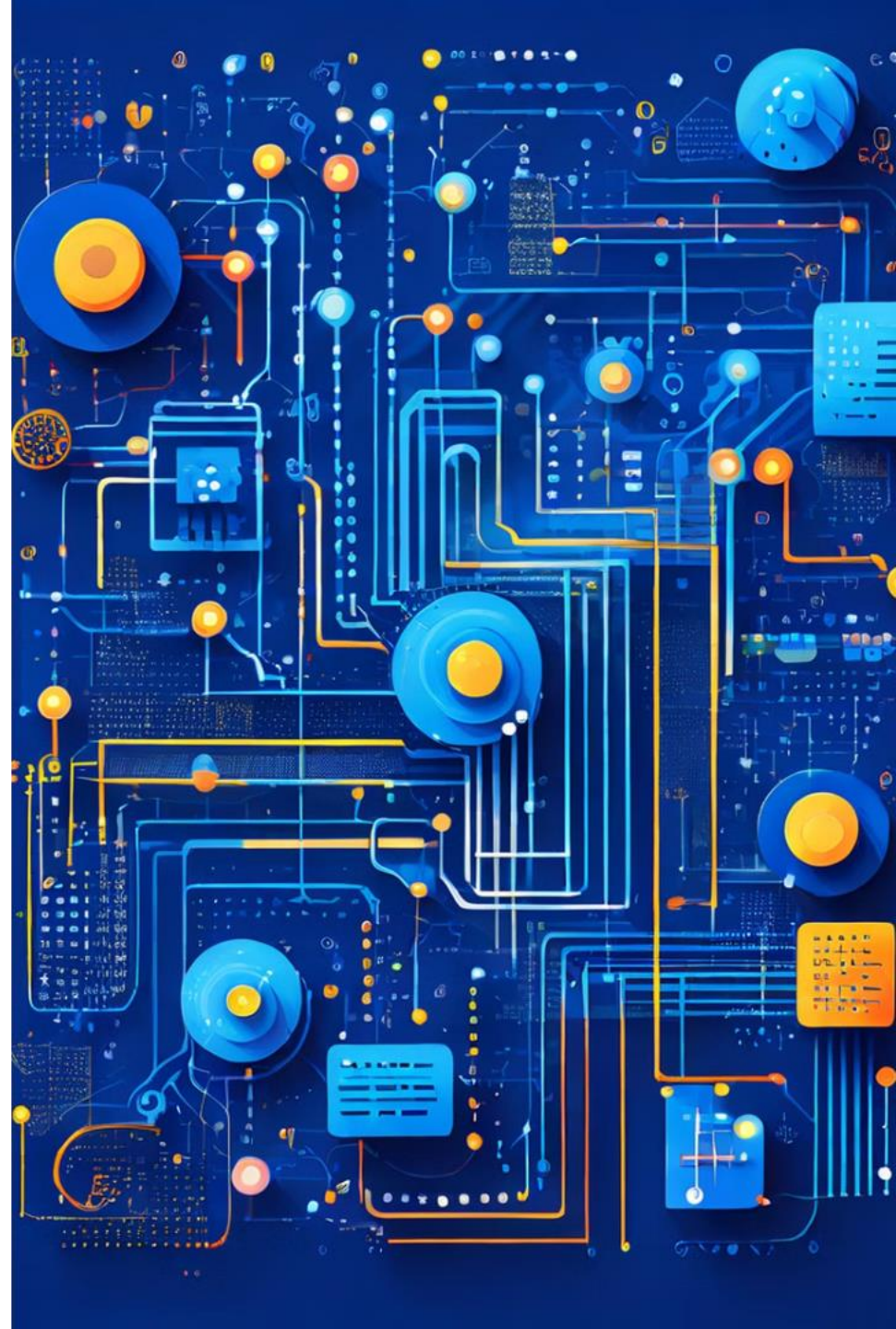
Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

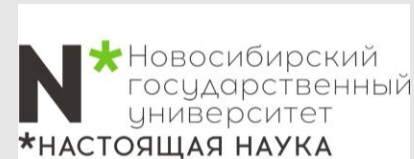


Давайте познакомимся



Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.



Адженда

**Жадные
алгоритмы**

45 минут

**Динамическое
программирова
ние**

45 минут

Адженда

**Жадные
алгоритмы**

45 минут

**Динамическое
программирова
ние**

45 минут

Введение

Дать точное определение жадного алгоритма достаточно трудно, если вообще возможно. Алгоритм называется жадным, если он строит решение за несколько мелких шагов, а решение на каждом шаге принимается без опережающего планирования, с расчетом на оптимизацию некоторого внутреннего критерия.

Когда жадный алгоритм успешно находит оптимальное решение нетривиальной задачи, этот факт обычно дает интересную и полезную информацию о структуре самой задачи.

Жадные алгоритмы — это класс алгоритмов, которые на каждом шаге делают локально оптимальный выбор в надежде, что эти локальные оптимумы приведут к глобальному оптимуму.

Примеры

- Кратчайшие пути в графе (след. семестр)
- Задача нахождения минимального остовного дерева (след. семестр)
- Реализация алгоритма Крускала (след. семестр)
- Коды Хаффмана и сжатие данных (рассматривали)

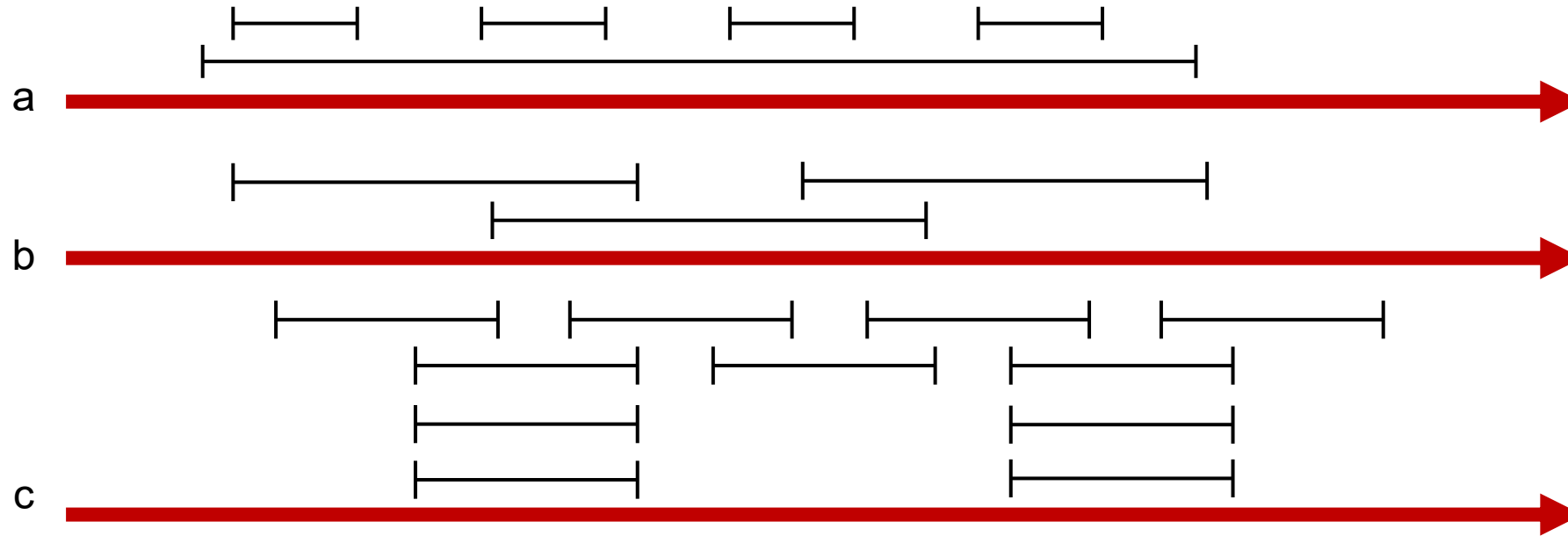
Сегодня рассмотрим еще несколько...



Интервальное планирование

Задача интервального планирования.

Имеется множество заявок $\{1, 2, \dots, n\}$; i -я заявка соответствует интервалу времени, который начинается в $s(i)$ и завершается в $f(i)$. Подмножество заявок называется совместимым, если никакие две заявки не перекрываются по времени; наша цель — получить совместимое подмножество как можно большего размера. Совместимые множества максимального размера называются **оптимальными**.



В случае **a** не работает выбор интервала с самым ранним начальным временем; в случае **b** не работает выбор самого короткого интервала; в случае **c** не работает выбор интервала с минимальным количеством конфликтов.

Проектирование жадного алгоритма

Определим алгоритм более формально. Пусть R — множество заявок, не принятых и не отклоненных на данный момент, а A — множество принятых заявок.

Инициализировать R множеством всех заявок, A — пустым множеством
 Пока множество R не пусто
 Выбрать заявку $i \in R$ с наименьшим конечным временем
 Добавить заявку i в A
 Удалить из R все заявки, несовместимые с запросом i
 Конец цикла
 Вернуть A как множество принятых заявок

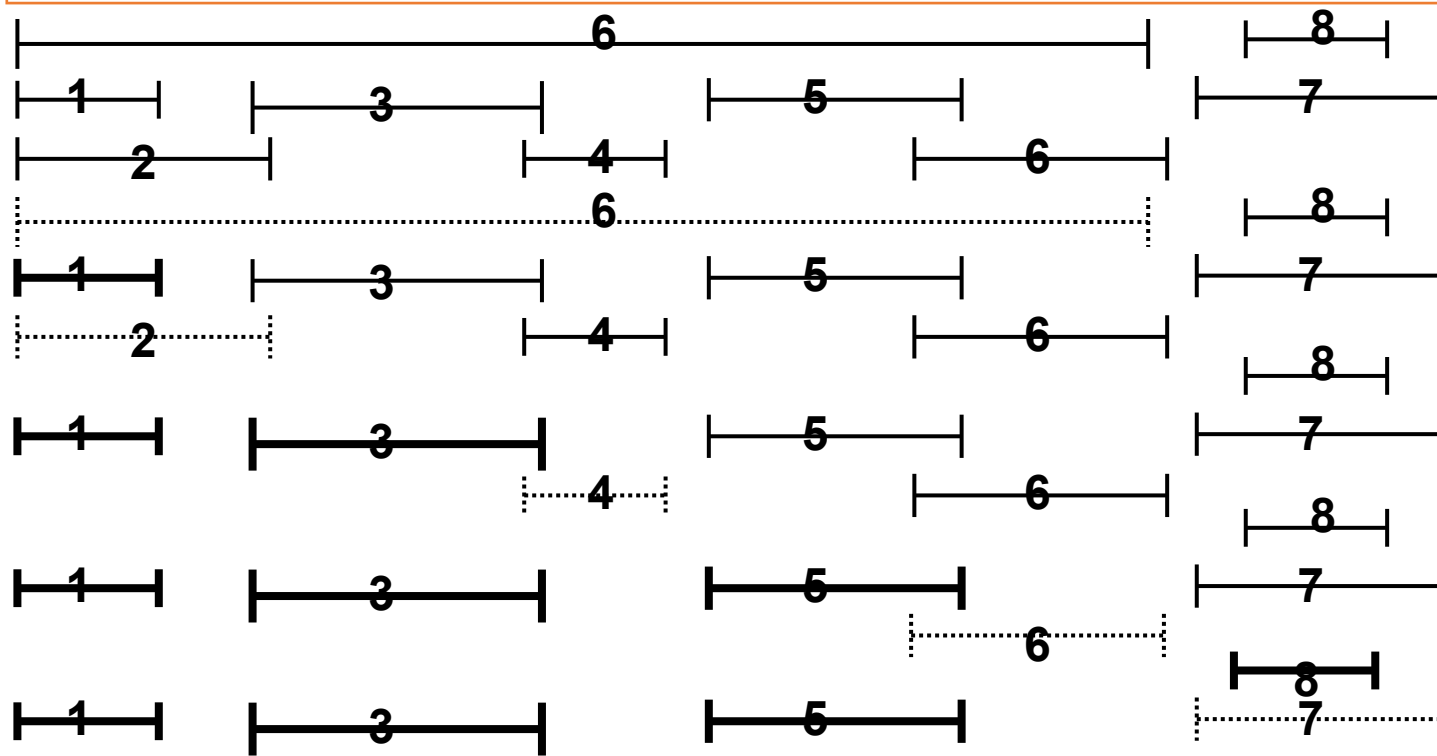
Интервалы, пронумерованные по порядку

Выбор интервала 1

Выбор интервала 3

Выбор интервала 5

Выбор интервала 8



Анализ алгоритма

Утверждение 16.1 Множество A состоит из совместимых заявок.

Продemonстрируем, что это решение **оптимально**.

Пусть O — оптимальное множество интервалов. В идеале хотелось бы показать, что $A = O$, но это уже слишком: оптимальных решений может быть несколько, и в лучшем случае A совпадает с одним из них. Покажем, что $|A| = |O|$, то есть A содержит столько же интервалов, сколько и O , а следовательно, является оптимальным решением.

Критерий, по которому наш жадный алгоритм опережает решение O .

Мы будем сравнивать частичные решения, создаваемые жадным алгоритмом, с начальными сегментами решения O и шаг за шагом покажем, что жадный алгоритм работает не хуже.

Пусть $i_1, \dots, i_k$ — множество заявок A в порядке их добавления в A . Заметьте, что $|A| = k$. Аналогичным образом множество заявок O будет обозначаться $j_1, \dots, j_m$. Мы намерены доказать, что $k = m$. Допустим, заявки в O также упорядочены слева направо по соответствующим интервалам, то есть по начальным и конечным точкам. Не забывайте, что заявки в O совместимы, то есть начальные точки следуют в том же порядке, что и конечные.

Может ли r -ий интервал жадного алгоритма кончаться позднее?



Анализ алгоритма

Выбор жадного метода происходит от желания освободить ресурс как можно раньше после удовлетворения первой заявки.

Утверждение 16.2 Для всех индексов $r \leq k$ выполняется условие $f(i_r) \leq f(j_r)$.

Доказательство. Метод индукции. Для $r = 1$ оно очевидным образом истинно: алгоритм начинается с выбора заявки i_1 с наименьшим временем завершения.

Теперь рассмотрим $r > 1$. Согласно гипотезе индукции, будем считать, что утверждение истинно для $r - 1$, и попробуем доказать его для r . Индукционная гипотеза позволяет считать, что $f(i_{r-1}) \leq f(j_{r-1})$. Если r -й интервал алгоритма не завершается раньше, он должен «отставать». Но это невозможно: вместо того, чтобы выбирать интервал с более поздним завершением, жадный алгоритм всегда имеет возможность выбрать j_r и таким образом реализовать шаг индукции.

Утверждение 16.3 Жадный алгоритм возвращает оптимальное множество A .

Доказательство. От обратного. Если множество A не оптимально, то оптимальное множество O должно содержать больше заявок, то есть $m > k$. Применяя (16.2) для $r = k$, получаем, что $f(i_k) \leq f(j_k)$. Так как $m > k$, в O должна существовать заявка j_{k+1} . Она начинается после завершения заявки j_k , а следовательно, после завершения i_k . Получается, что после удаления всех заявок, несовместимых с заявками $i_1, \dots, i_k$, множество возможных заявок R по-прежнему будет содержать j_{k+1} . Но тогда жадный алгоритм останавливается на заявке i_k , а должен останавливаться только на пустом множестве R , — противоречие. ■



Свойства алгоритма

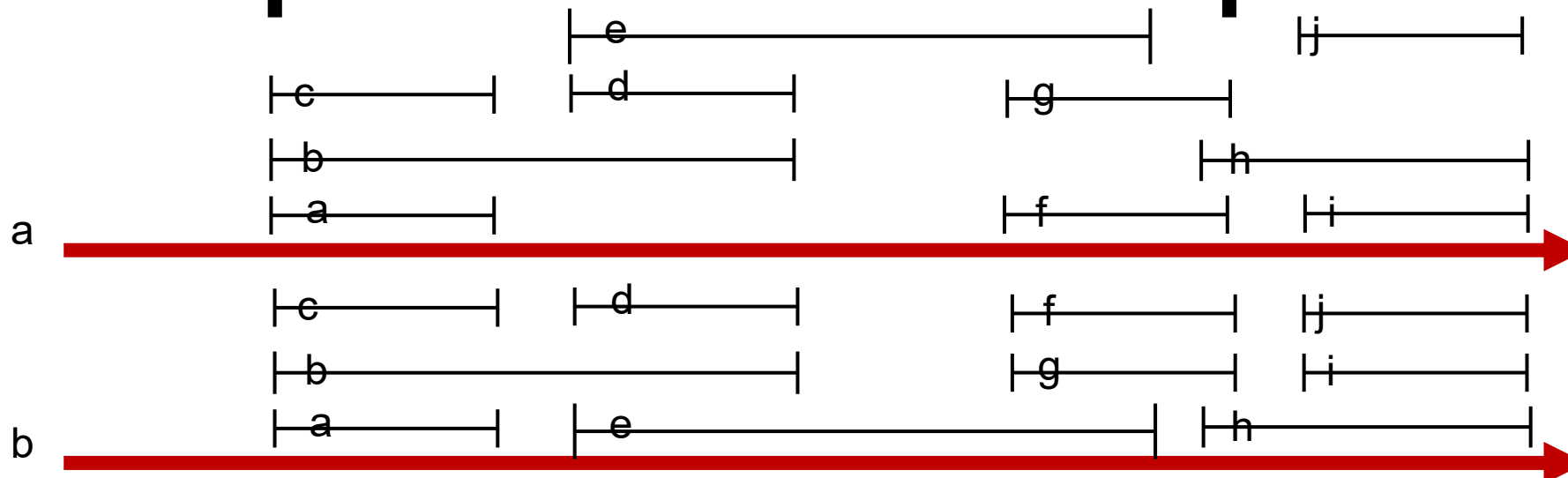
Скорость алгоритма: Алгоритм может выполняться за время $O(n \log n)$.

- Сначала n заявок следует отсортировать в порядке конечного времени и пометить их в этом порядке; соответственно предполагается, что $f(i) \leq f(j)$, когда $i < j$. Этот шаг выполняется за время $O(n \log n)$.
- За дополнительное время $O(n)$ строится массив $S[1 \dots n]$, в котором элемент $S[i]$ содержит значение $s(i)$.
- Теперь выбор заявок осуществляется обработкой интервалов в порядке возрастания $f(i)$. Сначала всегда выбирается первый интервал; затем интервалы перебираются по порядку до тех пор, пока не будет достигнут первый интервал j , для которого $s(j) \geq f(1)$; он также включается в результат. Эта часть алгоритма выполняется за время $O(n)$.

Расширения алгоритма

- Планировщик должен принимать решения по принятию или отклонению заявок до того, как ему будет известен полный набор заявок.
- Заявки обладают разной ценностью
- Пропускная способность > 1 .

Планирование всех интервалов



В задаче интервального планирования используется один ресурс и много заявок в форме временных интервалов; требуется выбрать, какие заявки следует принять, а какие отклонить. Во взаимосвязанной задаче используется несколько идентичных ресурсов, для которых необходимо распланировать все заявки с использованием минимально возможного количества ресурсов.

Для наглядности рассмотрим пример, а. Все заявки из этого примера могут быть распределены по трем ресурсам, b: заявки размещаются в три строки, каждая из которых содержит набор неперекрывающихся интервалов. В общем случае решение с k ресурсами можно представить как задачу распределения заявок в k строк с неперекрывающимися интервалами; первая строка содержит все интервалы, назначенные первому ресурсу, вторая строка — интервалы, назначенные второму ресурсу, и т. д.

Возможно ли обойтись только двумя ресурсами в этом конкретном примере? Разумеется, нет.

Планирование всех интервалов

Утверждение 16.4. В любой ситуации интервального разбиения количество необходимых ресурсов не меньше глубины множества интервалов.

Доказательство. Допустим, множество интервалов имеет глубину d , а интервалы $I_1, \dots, I_d$ проходят через одну общую точку временной шкалы. Все эти интервалы должны быть распределены по разным ресурсам, поэтому решению необходимы минимум d ресурсов.

Проектирование алгоритма Пусть d — глубина множества интервалов; каждому интервалу будет назначена метка из множества чисел $\{1, 2, \dots, d\}$ так, чтобы перекрывающиеся интервалы помечались разными числами. Так мы получим нужное решение.

Пусть $I_1, I_2, \dots, I_n$ — обозначения интервалов в указанном порядке

Для $j = 1, 2, 3, \dots, n$

Для каждого интервала I_i , который предшествует I_j в порядке сортировки и перекрывает его
Исключить метку I_i из рассмотрения для I_j

Конец цикла

Если существует метка из множества $\{1, 2, \dots, d\}$, которая еще не исключена
Присвоить неисключенную метку I_j

Иначе

Оставить I_j без метки (как обработка ошибки)

Конец Условия

Конец цикла

Анализ алгоритма

Утверждение 16.5. При использовании описанного жадного алгоритма каждому интервалу будет назначена метка, и никаким двум перекрывающимся интервалам не будет присвоена одна и та же метка.

Доказательство. Начнем с доказательства того, что ни один интервал не останется не помеченным. Рассмотрим один из интервалов I_j и предположим, что в порядке сортировки существует t интервалов, которые начинаются ранее и перекрывают его. Эти t интервалов в сочетании с I_j образуют множество из $t + 1$ интервалов, которые все проходят через общую точку временной шкалы (а именно начальное время I_j), поэтому $t + 1 \leq d$. Следовательно, $t \leq d - 1$. Из этого следует, что по крайней мере одна из меток d не будет исключена из этого множества интервалов t , поэтому существует метка, которая может быть назначена I_j .

Далее утверждается, что никаким двум перекрывающимся интервалам не будут назначены одинаковые метки. В самом деле, возьмем два перекрывающихся интервала I и I' и предположим, что I предшествует I' в порядке сортировки. Затем, при рассмотрении алгоритмом I' , интервал I принадлежит множеству интервалов, метки которых исключаются из рассмотрения; соответственно, алгоритм не назначит I' метку, которая использовалась для I . ■

Утверждение 16.6. Описанный жадный алгоритм связывает каждый интервал с ресурсом, используя количество ресурсов, равное глубине множества интервалов. Это количество ресурсов является минимально необходимым, то есть оптимальным.

Планирование для минимизации задержки: метод замены

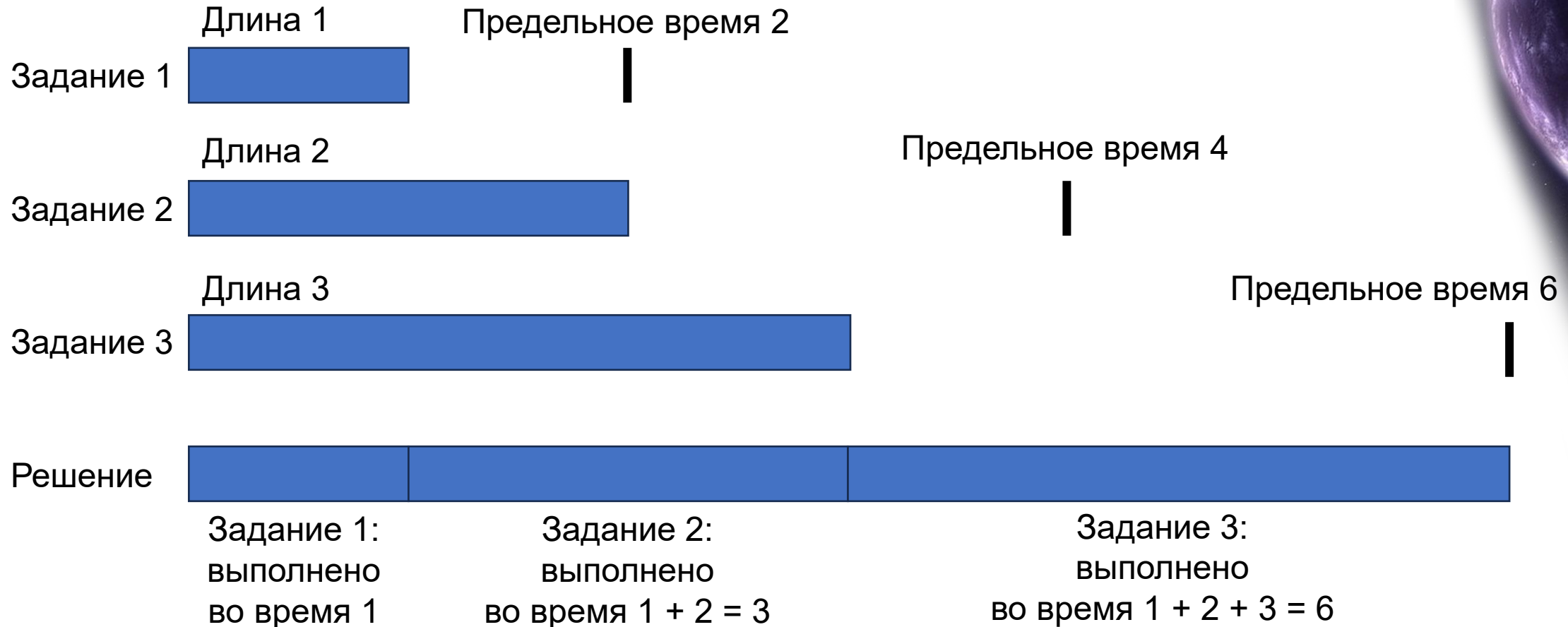
Задача

Вернемся к ситуации с одним ресурсом и набором из n заявок на его использование в течение интервала времени. Будем считать, что ресурс доступен, начиная с времени s . Однако в отличие от предыдущей задачи заявки становятся более гибкими — вместо начального и конечного времени заявка содержит предельное время d_i и непрерывный интервал времени длиной t_i , а начинаться она может в любое время до своего предельного времени. Каждой принятой заявке должен быть выделен интервал времени длиной t_i , и разным заявкам должны назначаться неперекрывающиеся интервалы.

Допустим, мы намерены удовлетворить каждую заявку, но некоторые заявки могут быть отложены на более позднее время. Таким образом, начиная с общего начального времени s , каждой заявке i выделяется интервал времени t_i ; обозначим этот интервал $[s(i), f(i)]$, где $f(i) = s(i) + t_i$. Однако в отличие от предыдущей задачи этот алгоритм должен определить начальное время (а следовательно, и конечное время) для каждого интервала.

Заявка i будет называться задержанной, если она не успевает завершиться к предельному времени, то есть если $f(i) > d_i$. Задержка такой заявки i определяется по формуле $l_i = f(i) - d_i$. Будем считать, что если заявка не является просроченной, то $l_i = 0$. Целью новой задачи оптимизации будет планирование всех заявок с неперекрывающимися интервалами, обеспечивающее минимизацию максимальной задержки $L = \max_i l_i$.

Планирование для минимизации задержки: метод замены



Пример планирования с минимизацией задержки

Проектирование алгоритма

Как же должен выглядеть жадный алгоритм для такой задачи?

Пусть просматриваем данные заданий (t_i, d_i) .

- Планирование заданий по возрастанию длины t_i для того, чтобы поскорее избавиться от коротких заданий. **Пример-опровержение**, с двумя заданиями: у первого $t_1 = 1$ и $d_1 = 11$, а у второго $t_2 = 10$ и $d_2 = 10$. Для достижения задержки $L = 0$ второе задание должно начаться немедленно.
- Ориентирование на задания с минимальной задержкой $d_i - t_i$, таким образом, более естественный жадный алгоритм должен сортировать задания в порядке возрастания запаса времени $d_i - t_i$. **Пример-опровержение**, $t_1 = 1$ и $d_1 = 2$, а у второго $t_2 = 10$ и $d_2 = 10$.
- Задания просто сортируются в порядке возрастания предельного времени d_i и планируются в этом порядке. Алгоритм:

Упорядочить задания по предельному времени

Для простоты считается, что $d_1 \leq \dots \leq d_n$

В исходном состоянии $f = s$

Рассмотреть задания $i = 1, \dots, n$ в таком порядке

Назначить задание i временному интервалу от $s(i) = f$ до $f(i) = f + t_i$ Присвоить $f = f + t_i$

Конец

Вернуть множество спланированных интервалов $[s(i), f(i)]$ для $i = 1, \dots, n$



Анализ алгоритма

Утверждение 16.7. Существует оптимальное расписание без времени простоя.

Рассмотрим оптимальное расписание O . Нашей целью будет постепенная модификация O , с сохранением оптимальности на каждом шаге и преобразованию его в расписание, идентичное A , найденному жадным алгоритмом.

Если в нём есть промежуток простоя $[t_1, t_2)$, то сдвинем все задания, начинающиеся после t_2 , влево на $(t_2 - t_1)$. Времена завершения всех заданий либо уменьшаются, либо остаются прежними $\Rightarrow$ максимальная задержка не увеличивается.

Обозначим A' содержит инверсию, если задание i с d_i предшествует заданию j с $d_j < d_i$.

Утверждение 16.8. Все расписания, не содержащие инверсий и простоев, обладают одинаковой максимальной задержкой.

Доказательство. Если два разных расписания не содержат ни инверсий, ни простоев, то задания в них могут следовать в разном порядке, **различаться будет только порядок заданий с одинаковым предельным временем**. Рассмотрим такое предельное время d . В обоих расписаниях задания с предельным временем d планируются последовательно (после всех заданий с более ранним предельным временем и до всех заданий с более поздним предельным временем). **Среди всех заданий с предельным временем d последнее обладает наибольшей задержкой**, причем эта задержка не зависит от порядка заданий. ■

Анализ алгоритма

Утверждение 16.9. Существует оптимальное расписание, не имеющее инверсий и времени простоя.

Доказательство. Согласно (16.7), существует оптимизация расписание O без времени простоя. Доказательство будет состоять из серии утверждений, первое из которых доказывается легко.

(a) Если O содержит инверсию, то $\exists$ пара заданий i и j , для которых j следует в расписании немедленно после i и $d_j < d_i$.

Рассмотрим инверсию, в которой задание a стоит в расписании до задания b и $d_a > d_b$. При перемещении в порядке планирования заданий от a к b где-то встретится точка, в которой предельное время уменьшается в первый раз. Она соответствует паре последовательных заданий, образующих инверсию.

Теперь допустим, что O содержит как минимум одну инверсию, и в соответствии с (a) пусть i и j образуют пару инвертированных запросов, занимающих последовательные позиции в расписании. Меняя местами заявки i и j в расписании O , мы уменьшим количество инверсий в O на 1. Пара (i, j) создавала инверсию в O , перестановка эту инверсию устраняет, и новые инверсии при этом не создаются. Таким образом, **(b)** После перестановки i и j образуется расписание, содержащее на 1 инверсию меньше.

В этом доказательстве труднее всего обосновать тот факт, что расписание после устранения инверсии также является оптимальным.

Анализ алгоритма

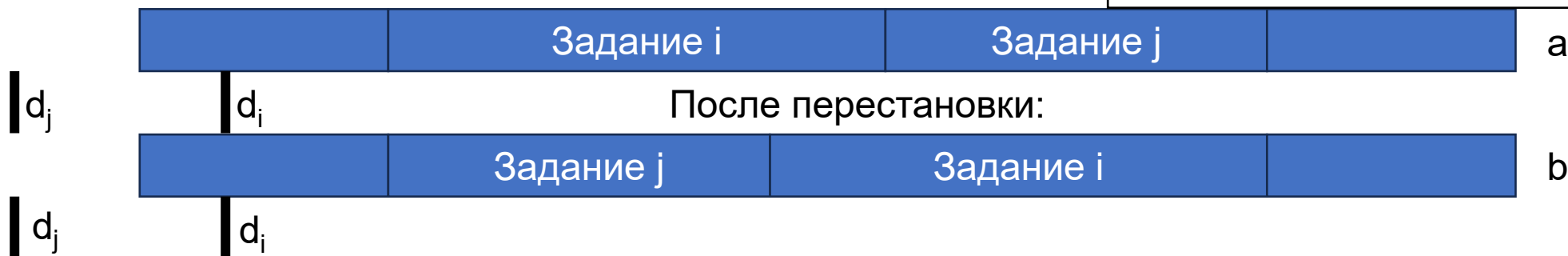
(с) Максимальная задержка в новом расписании, полученном в результате перестановки, не превышает максимальной задержки O . Очевидно, что если удастся доказать (с), то задачу можно считать выполненной.

Исходное расписание O может иметь не более $\binom{n}{2}$ инверсий (если инвертированы все пары), после максимум $\binom{n}{2}$ перестановок мы получим оптимальное расписание без инверсий. ■

Доказательство (с). Введем вспомогательное обозначение для описания расписания O : предположим, каждая заявка r планируется на интервал времени $[s(r), f(r)]$ и обладает задержкой l'_r . Пусть $L' = \max_r l'_r$ обозначает максимальную задержку этого расписания. Обозначим расписание с перестановкой $\bar{O}$; обозначения $\bar{s}(r), \bar{f}(r), \bar{l}_r$ и $\bar{L}$ будут представлять соответствующие характеристики расписания с перестановкой.

До перестановки:

Перестановка влияет только на время завершения i и j



Время завершения j до перестановки в точности равно времени завершения i после перестановки. Таким образом, все задания, кроме i и j , в двух расписаниях завершаются одновременно. Кроме того, перестановка не увеличивает задержку задания j .

Анализ алгоритма

Следовательно, остается беспокоиться только о задании i : его задержка могла увеличиться. После перестановки задание i завершается во время $f(j)$, в которое завершалось задание j в расписании O . Если задание i «опаздывает» в новом расписании, его задержка составляет $\bar{l}_i = \bar{f}(i) - d_i = f(j) - d_i$. Но здесь принципиально то, что i не может задерживаться в расписании $\bar{O}$ больше, чем задерживалось задание j в расписании O . А конкретнее, из нашего предположения $d_i > d_j$ следует, что

$$\bar{l}_i = \bar{f}(i) - d_i < f(j) - d_j = l'_j.$$

Так как задержка расписания O была равна $L \geq l'_j \geq \bar{l}_i$, из этого следует, что перестановка не увеличивает максимальную задержку расписания. ■

Утверждение 16.10. Расписание A , построенное жадным алгоритмом Джексона, обеспечивает оптимальную максимальную задержку L .

Доказательство. Пункт (16.9) доказывает, что оптимальное расписание без инверсий существует. С другой стороны, согласно (16.8), все расписания без инверсий имеют одинаковую максимальную задержку, так что расписание, построенное жадным алгоритмом, является оптимальным. ■

Расширения

Заявки i , которые, в дополнение к предельному времени d_i и запрашиваемому времени t_i также содержат минимально возможное начальное время r_i (называется временем разблокировки). Пример: «Можно ли зарезервировать аудиторию для проведения двухчасовой лекции в интервале от 13 до 17 часов?»

Оптимальное кэширование: более сложный пример замены

Общим термином «кэширование» называется процесс хранения малых объемов данных в быстрой памяти, чтобы уменьшить время, которое тратится на взаимодействия с медленной памятью.

Задача

Рассматривается множество U из n фрагментов данных, хранящихся в основной памяти. Есть быстрая память — кэш, способная в любой момент времени хранить $k < n$ фрагментов данных. Мы получаем

последовательность элементов данных $D = d_1, d_2, \dots, d_m$ из U — это последовательность обращений к памяти, которую требуется обработать; при обработке следует постоянно принимать решение о том, **какие k элементов должны храниться в кэше**. Запрашиваемый элемент d_i очень быстро читается, если он уже находится в кэше; в противном случае ситуация называется кэш-промахом; естественно, мы стремимся к тому, чтобы количество промахов было сведено к минимуму.

Аналогия и иерархия

Цель: Визуализировать "Slow/Large" и "Fast/Small"

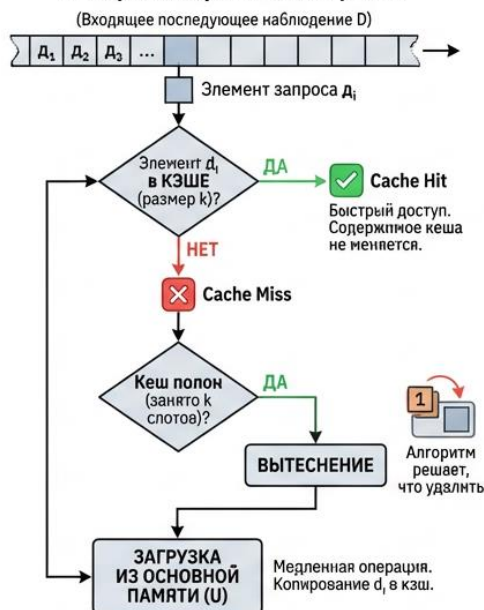


Абстрактная модель и алгоритм

Цель: Show формального решения-making процесс.

Алгоритм обработки запросов

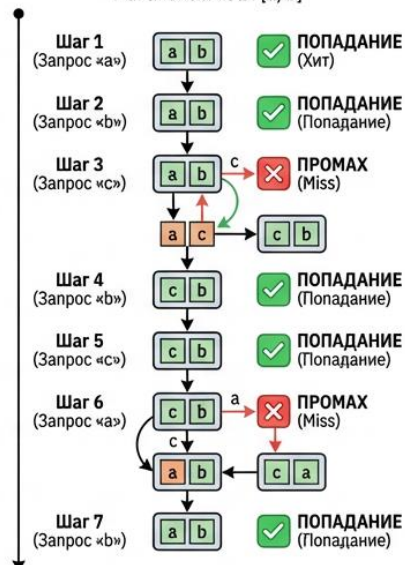
(Входящее последующее наблюдение D)



Пример исполнения

Цель: Illustrate a specific trace.

Трассировка примера
Данные $U: \{a, b, c\}$, Размер кэша $k=2$.
Начальный кэш: $[a, b]$



Разработка и анализ алгоритма

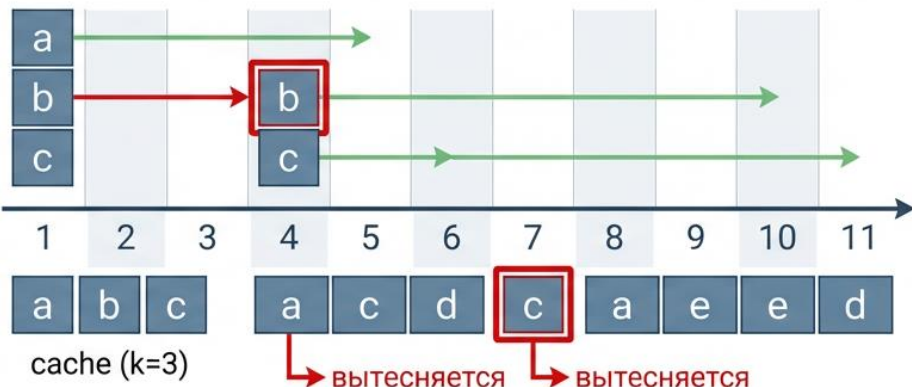
В 1960-х годах Лес Белади показал, что следующее простое правило всегда приводит к минимальному количеству промахов:

Когда элемент d_i должен быть внесен в кэш, вытеснить элемент, который будет использоваться позднее всех остальных - «алгоритм отдаленного использования».

Утверждение 16.11. Сокращенный план $\bar{S}$ заносит в кэш не больше элементов, чем план S . Заметьте, что для каждого сокращенного плана количество элементов, включаемых в кэш, точно совпадает с количеством промахов.

Альтернативный план S' (тоже оптимальный)

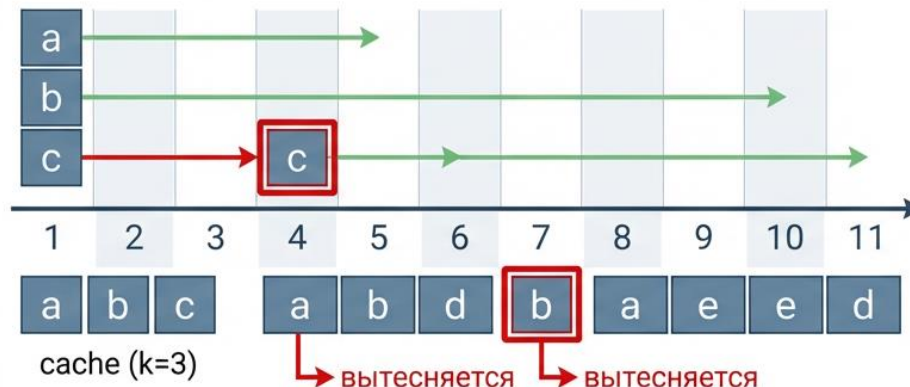
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a \rightarrow d \rightarrow e \rightarrow a \rightarrow d \rightarrow b \rightarrow c$



5 промахов

Алгоритм Белади (оптимальный)

$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a \rightarrow d \rightarrow e \rightarrow a \rightarrow d \rightarrow b \rightarrow c$

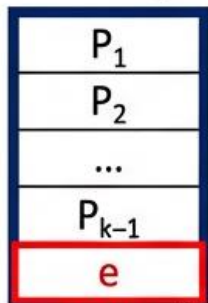


5 промахов

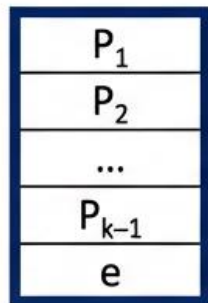
Доказательство оптимальности алгоритма отдаленного использования

Общий случай до преобразования (шаг $j+1$)

Кэш алгоритма
Far Future (SFF)



Кэш произвольного плана S (до
этого момента совпадал с SFF)

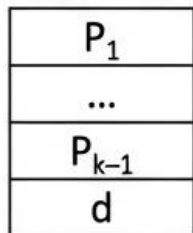


Приходит запрос
на отсутствующую
страницу d
(промах)

e — жертва SFF: самое
дальнее будущее использование

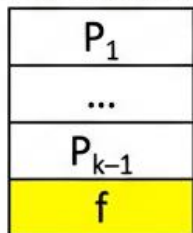
→ SFF вытесняет e
→ добавляет d

Новый кэш:



→ S вытесняет другую
страницу $f \neq e$
(f обведена жёлтым)

Новый кэш:



Кэши разошлись:
в S осталась f , в SFF — d

Рассмотрим произвольную последовательность обращений к памяти D ; пусть S_{FF} обозначает план, построенный алгоритмом отдаленного использования, а S^* — план с минимально возможным количеством промахов.

Утверждение 16.12. Пусть S — сокращенный план, который принимает те же решения по вытеснению, что и S_{FF} , в первых j элементах последовательности для некоторого j . Тогда существует сокращенный план S' , который принимает те же решения, что и S_{FF} , в первых $j+1$ элементах и создает не больше промахов, чем S .

Доказательство. Рассмотрим $(j+1)$ -е обращение к элементу $d = d_{j+1}$. Так как S и S_{FF} до этого момента были согласованы, содержимое кэша в них не различается. Если d находится в кэше в обоих планах, то решения по вытеснению не требуются, так что S согласуется с S_{FF} на шаге $j+1$, и, следовательно, $S' = S$. Аналогичным образом, если элемент d должен быть включен в кэш, но S и S_{FF} вытеснят один и тот же элемент, чтобы освободить место для d , и снова $S' = S$.

Доказательство оптимальности алгоритма отдаленного использования

Итак, на шаге $j + 1$ интересная ситуация возникает, когда d необходимо добавить в кэш, причем S вытесняет f , а S_{FF} вытесняет $e \neq f$. Здесь S и S_{FF} не согласуются к шагу $j + 1$.

Сначала нужно сделать так, чтобы план S' вытеснял e вместо f . Затем нужно убедиться, что количество промахов у S' не больше S . Мы должны привести кэш S' в такое же состояние, как у S , не создавая ненужных промахов. Когда содержимое кэша будет совпадать, можно будет завершить построение S' , просто повторяя поведение S . От запроса $j + 2$ и далее S' ведет себя в точности как S , пока впервые не будет выполнено одно из следующих условий:

- (i) **Происходит обращение к элементу $g \neq e, f$** , который не находится в кэше S , и S вытесняет e , чтобы освободить для него место. Так как S' и S различаются только в e и f , это означает, что g также не находится в кэше S' ; тогда из S' вытесняется f и кэши S и S' совпадают. *В оставшейся части последовательности S' ведет себя точно так же, как S .*
- (ii) **Происходит обращение к f , и S вытесняет элемент e'** . Если $e' = e$, все просто: S' просто обращается к f из кэша, и после этого шага кэши S и S' совпадают. Если $e' \neq e$, то S' также вытесняет e' и добавляет e из основной памяти; в результате S и S' тоже имеют одинаковое содержимое кэша. Однако необходима осторожность, так как элемент e переносится в кэш до того, как в нем возникнет прямая необходимость. Итак, чтобы завершить эту часть построения, мы преобразуем S' в сокращенную форму $\bar{S}'$, используя (16.11); количество элементов, включаемых S' , при этом не увеличивается, и S' по-прежнему согласуется с S_{FF} на шаге $j + 1$.

Доказательство оптимальности алгоритма отдаленного использования

После одного шага замены $\rightarrow$ план S'

SFF: как и раньше,
вытеснил e

P_1
P_2
...
P_{k-1}
e
d

Новый план S' (полученный
из S преобразованием по
лемме 1.12): на этом же
шаге $j+1$ тоже вытесняет e
(а не f !)

P_1
...
...
P_{k-1}
d

S' теперь совпадает с SFF на первых $j+1$ запросах
и имеет $\leq$ число промахов, чем исходный S
(дальше S' будет копировать S до момента
«схлопывания» разницы e/f без лишних промахов)

Итак, в обоих случаях мы получаем новый сокращенный план S' , который согласуется с S_{FF} на первых $j+1$ элементах и обеспечивает не большее количество промахов, чем S . И здесь принципиально то, что один из этих двух случаев возникнет до обращения к e . Дело в том, что на шаге $j+1$ алгоритм отдаленного использования вытеснил элемент (e), который понадобится в самом отдаленном будущем; следовательно, обращению к e должно предшествовать обращение к f , и тогда возникнет ситуация (ii). ■

Этот результат позволяет легко завершить доказательство оптимальности. Мы начнем с оптимального плана S^* и воспользуемся (16.12) для построения плана S_1 , который согласуется с S_{FF} на первом шаге. Затем мы продолжаем применять (16.12) индуктивно для $j = 1, 2, 3, \dots, m$, строя планы S_j , согласующиеся с S_{FF} на первых j шагах. Каждый план не увеличивает количество промахов по сравнению с предыдущим, а по определению $S_m = S_{FF}$, поскольку планы согласуются на всей последовательности.

Таким образом, получаем: **утверждение 16.13** S_{FF} порождает не больше промахов, чем любой другой план следовательно, является оптимальным.

Кэширование в реальных рабочих условиях

Как упоминалось в предыдущем подразделе, оптимальный алгоритм Беладии предоставляет контрольную точку для оценки эффективности кэширования; однако на практике решения по вытеснению должны приниматься «на ходу», без информации о будущих обращениях.

Эксперименты показали, что лучшие алгоритмы кэширования в описанной ситуации представляют собой вариации на тему принципа **LRU (Least-RecentlyUsed)**, по которому из кэша вытесняется элемент с наибольшим временем, прошедшим с момента последнего обращения.

Если задуматься, речь идет о том же алгоритме Беладии с измененным направлением времени, — только наибольший продолжительный промежуток времени относится к прошлому, а не к будущему. Он эффективно работает, потому что для приложений обычно характерна локальность обращений: выполняемая программа, как правило, продолжает обращаться к тем данным, к которым она уже обращалась ранее. По этой причине в кэше обычно хранятся данные, которые использовались недавно. Спустя долгое время после практического принятия алгоритма LRU **Слитор и Тарьян** показали, что для эффективности LRU можно предоставить теоретический анализ, ограничивающий количество промахов относительно алгоритма отдаленного использования.



Адженда

**Жадные
алгоритмы**

45 минут

**Динамическое
программирова
ние**

45 минут

От жадных алгоритмов к динамическому программированию

Все предыдущие задачи имели работающий жадный алгоритм

- В реальности так бывает редко
- Для большинства задач естественного жадного решения просто не существует
- Когда жадность не работает — нужен более мощный инструмент

Динамическое программирование (ДП) — следующий уровень базируется на принципе «разделяй и властвуй»

- Действует противоположно жадности:
 - а. не делает необратимых выборов
 - б. неявно исследует всё пространство решений
- Разбивает задачу на пересекающиеся подзадачи
- Строит оптимальное решение из решений подзадач возрастающего размера
- Работает «в опасной близости» от полного перебора, но избегает экспоненциальной сложности за счёт кэширования

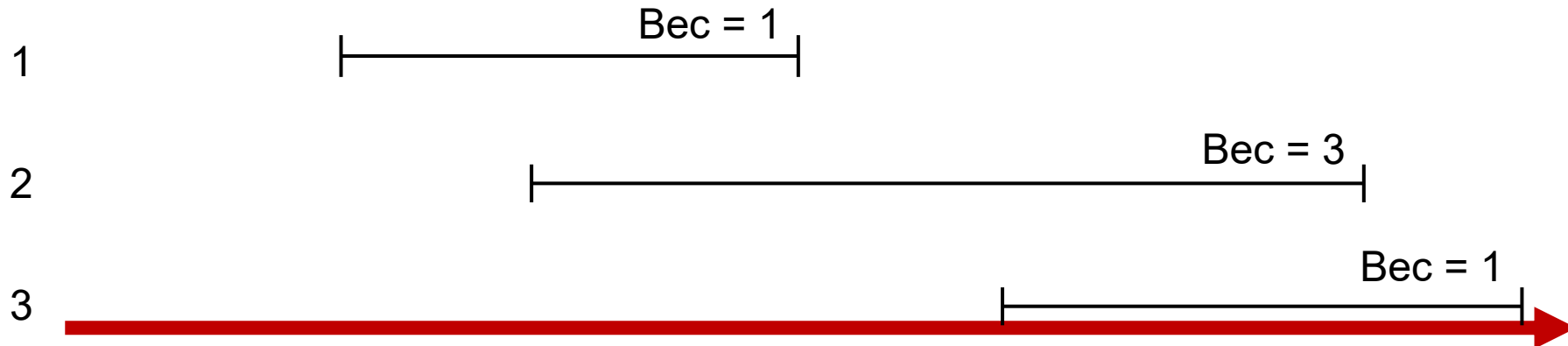
Примеры задач:

- Вычисление n-го числа Фибоначчи
- Задача о рюкзаке
- Оптимальный порядок умножения матриц
- Наибольшая возрастающая подпоследовательность

Взвешенное интервальное планирование

Задача взвешенного интервального планирования имеет более общий характер: с каждым интервалом связывается определенное значение (вес), а решением задачи считается множество с максимальным суммарным весом. Действительно, естественный жадный алгоритм для этой задачи неизвестен, что заставляет нас переключиться на динамическое программирование.

Индекс Простой пример задачи взвешенного интервального планирования



Имеются n заявок с пометками $1, \dots, n$; для каждой заявки i указывается начальное время s_i и конечное время f_i . С каждым интервалом i связывается некоторое значение — вес v_i . Два интервала называются совместимыми, если они не перекрываются.

Цель задачи: найти подмножества $S \subseteq \{1, \dots, n\}$ взаимно совместимых интервалов, максимизирующего сумму весов выбранных интервалов $\sum_{i \in S} v_i$.

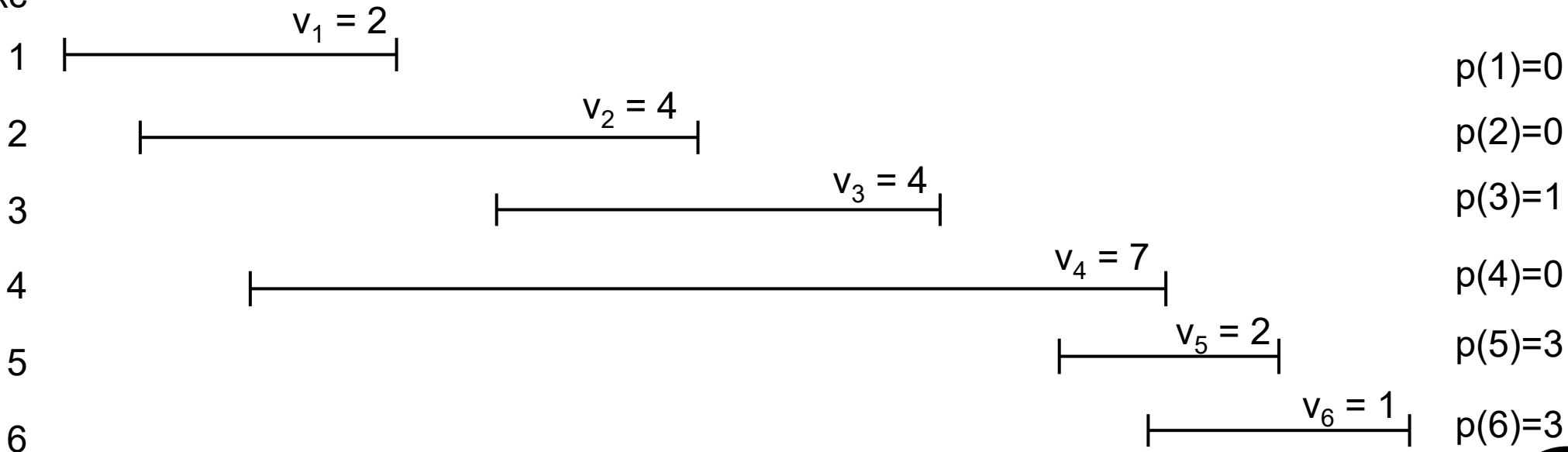
Разработка рекурсивного алгоритма

Предположим, заявки отсортированы в порядке неубывания конечного времени: $f_1 \leq f_2 \leq \dots \leq f_n$. Заявка i называется предшествующей заявке j , если $i < j$. Таким образом определяется естественный порядок «слева направо», в котором будут рассматриваться интервалы.

Определим $p(j)$ для интервала j как наибольший индекс $i < j$, при котором интервалы i и j не перекрываются. Другими словами, i — крайний левый интервал, который завершается до начала j . Определим $p(j) = 0$, если не существует заявки $i < j$, не перекрывающейся с j .

Задача взвешенного интервального планирования с функциями $p(j)$, определенными для каждого интервала j

Индекс



Разработка рекурсивного алгоритма

Рассмотрим неизвестное **оптимальное решение** O для интервалов $1..n$

Два взаимоисключающих случая:

- Последний интервал n входит в O
- Интервал n не входит в O

Случай 1: $n \in O$

- Все интервалы от $p(n) + 1$ до $n-1$ пересекаются с $n \rightarrow$ их нельзя брать
- $O \cap \{1, \dots, p(n)\}$ должно быть оптимальным для подзадачи $\{1, \dots, p(n)\}$
- Иначе можно улучшить O , заменив эту часть $\rightarrow$ противоречие оптимальности
- **Следовательно:** $OPT(n) = v_n + OPT(p(n))$

Случай 2: $n \notin O$

- O полностью лежит в $\{1, \dots, n-1\}$
- O должно быть оптимальным для подзадачи $\{1, \dots, n-1\}$
- Иначе можно улучшить $\rightarrow$ противоречие
- **Следовательно:** $OPT(n) = OPT(n-1)$

Так как других вариантов быть не может ($j \in O_j$ или $j \notin O_j$), можно утверждать, что

Утверждение 16.14. $OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$.

Утверждение 16.15. Заявка j принадлежит оптимальному решению для множества $\{1, 2, \dots, j\}$ в том и только в том случае, если $v_j + OPT(p(j)) \geq OPT(j-1)$.

Разработка рекурсивного алгоритма

Утверждение 16.16. $\text{Compute-Opt}(j)$ правильно вычисляет $\text{OPT}(j)$ для всех $j = 1, 2, \dots, n$.

Доказательство. По определению $\text{OPT}(0) = 0$. Теперь возьмем некоторое значение $j > 0$ и предположим, что $\text{Compute-Opt}(i)$ правильно вычисляет $\text{OPT}(i)$ для всех $i < j$. Из индукционной гипотезы следует, что $\text{Compute-Opt}(p(j)) = \text{OPT}(p(j))$ и $\text{Compute-Opt}(j - 1) = \text{OPT}(j - 1)$; следовательно, из (16.14)

$\text{OPT}(j) = \max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j - 1))$
 $= \text{Compute-Opt}(j)$ ■

Рекурсивный алгоритм для вычисления $\text{OPT}(n)$:

Compute-Opt(j)

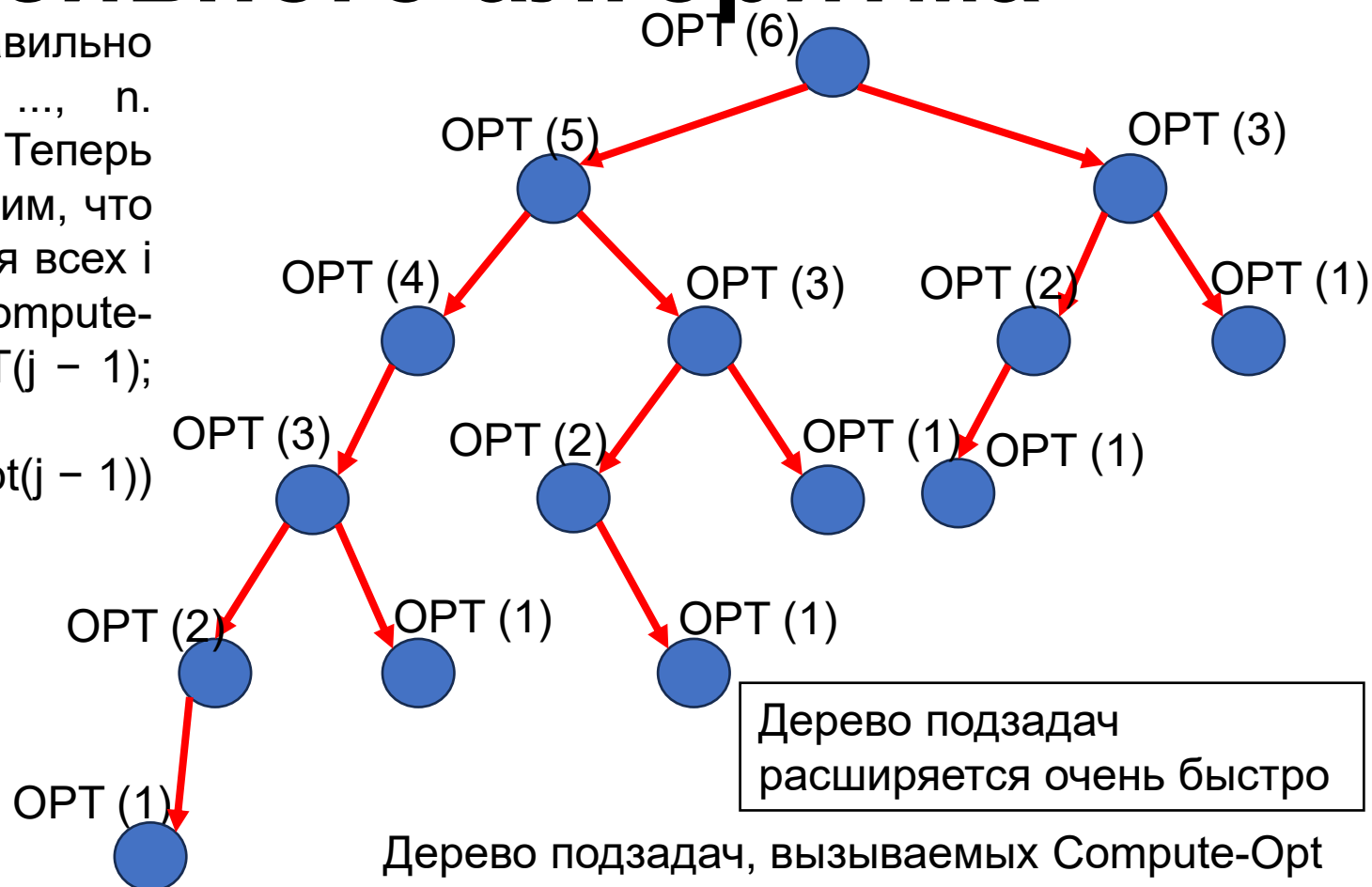
Если $j = 0$

Вернуть 0

Иначе

Вернуть $\max(v_j + \text{Compute-Opt}(p(j)), \text{Compute-Opt}(j - 1))$

Конец условия

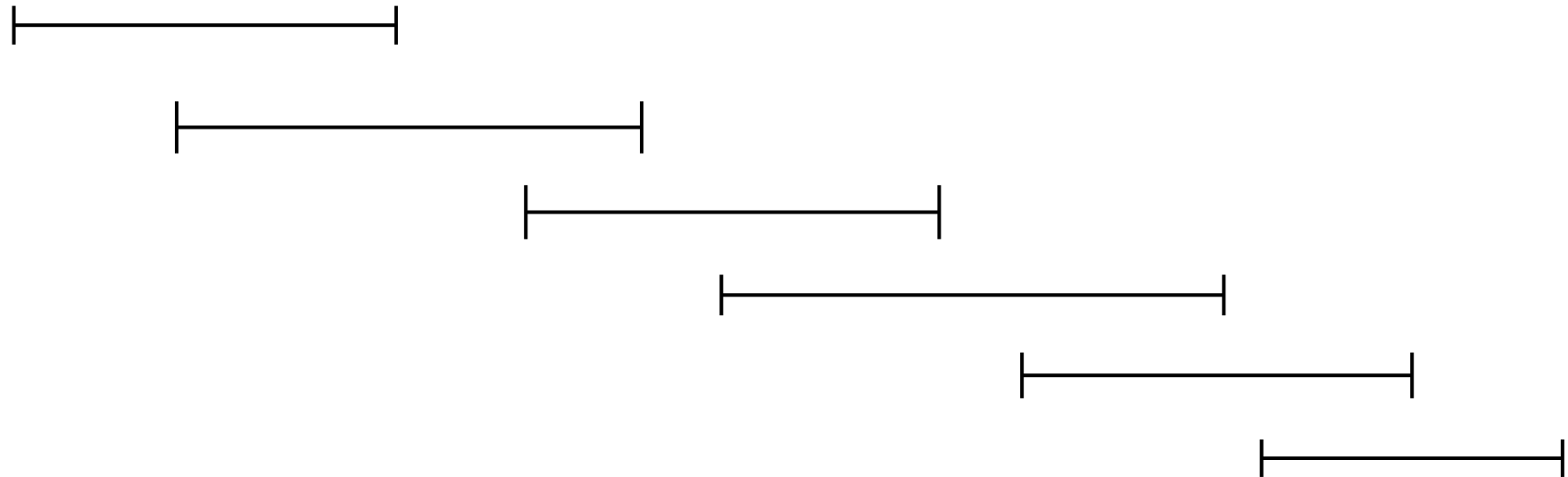


Дерево подзадач
расширяется очень быстро

Дерево подзадач, вызываемых Compute-Opt
 К сожалению, если реализовать алгоритм Compute-Opt в этом виде, его выполнение в худшем случае займет экспоненциальное время.

Разработка рекурсивного алгоритма

Или более экстремальный пример: в системе с частым наслоением, где $p(j) = j - 2$ для всех $j = 2, 3, 4, \dots, n$, мы видим, что `Compute-Opt(j)` генерирует отдельные рекурсивные вызовы для задач с размерами $j - 1$ и $j - 2$. Другими словами, общее количество вызовов `Compute-Opt` в этой задаче будет расти в темпе чисел Фибоначчи, что означает экспоненциальный рост. Таким образом, решение с полиномиальным временем так и остается нереализованным.



Экземпляр задачи взвешенного интервального планирования, для которого простая рекурсия `Compute-Opt` занимает экспоненциальное время. Веса интервалов в этом примере равны 1

Мемоизация рекурсии

Как устранить избыточность вызовов функций? Можно сохранить значение `Compute-Opt` в глобально-доступном месте при первом вычислении и затем просто использовать заранее вычисленное значение вместо всех будущих рекурсивных вызовов. Метод сохранения ранее вычисленных значений называется мемоизацией (memoization).

M-Compute-Opt(j)

Если $j = 0$

Вернуть 0

Иначе Если $M[j]$ не пусто

Вернуть $M[j]$

Иначе

Определить $M[j] = \max(v_j + M\text{-Compute-Opt}(p(j)), M\text{-Compute-Opt}(j - 1))$

Вернуть $M[j]$

Конец условия

Утверждение 16.17. Время выполнения `M-Compute-Opt(n)` равно $O(n)$ (предполагается, что входные интервалы отсортированы по конечному времени).

Доказательство. Время, потраченное на один вызов `M-Compute-Opt`, равно $O(1)$ без учета времени, потраченного в порожденных им рекурсивных вызовах. Количество «непустых» элементов M в исходном состоянии оно равно 0; но при каждом переходе на новый уровень с выдачей двух рекурсивных вызовов `M-Compute-Opt` заполняется новый элемент, а следовательно, количество заполненных элементов увеличивается на 1. Так как M содержит только $n + 1$ элемент, время выполнения `M-ComputeOpt(n)` равно $O(n)$, как и требовалось. ■

Вычисление решения помимо его значения

Алгоритм **M-ComputeOpt** легко расширяется для отслеживания оптимального решения: можно создать дополнительный массив S , в элементе $S[i]$ которого хранится оптимальное множество интервалов из $\{1, 2, \dots, i\}$. Однако наивное расширение кода для сохранения решений в массиве S увеличит время выполнения с дополнительным множителем $O(n)$: хотя позиция в массиве M может обновляться за время $O(1)$, запись множества в массив S занимает время $O(n)$. Чтобы избежать возрастания $O(n)$, вместо явного хранения S можно восстановить оптимальное решение по значениям, хранящимся в массиве M после вычисления оптимального значения. Из (16.15) известно, что j принадлежит оптимальному решению для множества интервалов $\{1, \dots, j\}$ в том, и только в том случае, если $v_j + \text{OPT}(p(j)) \geq \text{OPT}(j - 1)$.

Find-Solution(j)

Если $j = 0$

Ничего не выводить

Иначе Если $v_j + M[p(j)] \geq M[j - 1]$

Вывести j вместе с результатом **Find-Solution**($p(j)$)

Иначе

Вывести результат **Find-Solution**($j - 1$)

Конец условия

Утверждение 16.18. Для массива M с оптимальными значениями подзадач **Find-Solution** возвращает оптимальное решение за время $O(n)$.

Принципы динамического программирования: мемоизация или итерации с подзадачами

Теперь мы воспользуемся алгоритмом задачи взвешенного интервального планирования для обобщения базовых принципов динамического программирования. Кроме того, он поможет понять принцип, который играет ключевую роль: итерации с подзадачами вместо рекурсивного вычисления решений.

Ключом к построению эффективного алгоритма является массив M . В нем закодирована идея о том, что мы используем значение оптимальных решений подзадач для интервалов $\{1, 2, \dots, j\}$ по всем j , и он использует (16.14) для определения значения $M[j]$ на основании значений предшествующих элементов массива. Как только мы получаем массив M , задача решена.

Важно понять, что элементы M можно напрямую вычислять итеративным алгоритмом вместо рекурсии с мемоизацией. Просто начнем с $M[0] = 0$ и будем увеличивать j ; каждый раз, когда потребуется определить значение $M[j]$, ответ предоставляется (16.14). Алгоритм выглядит так:

Iterative-Compute-Opt

$M[0] = 0$

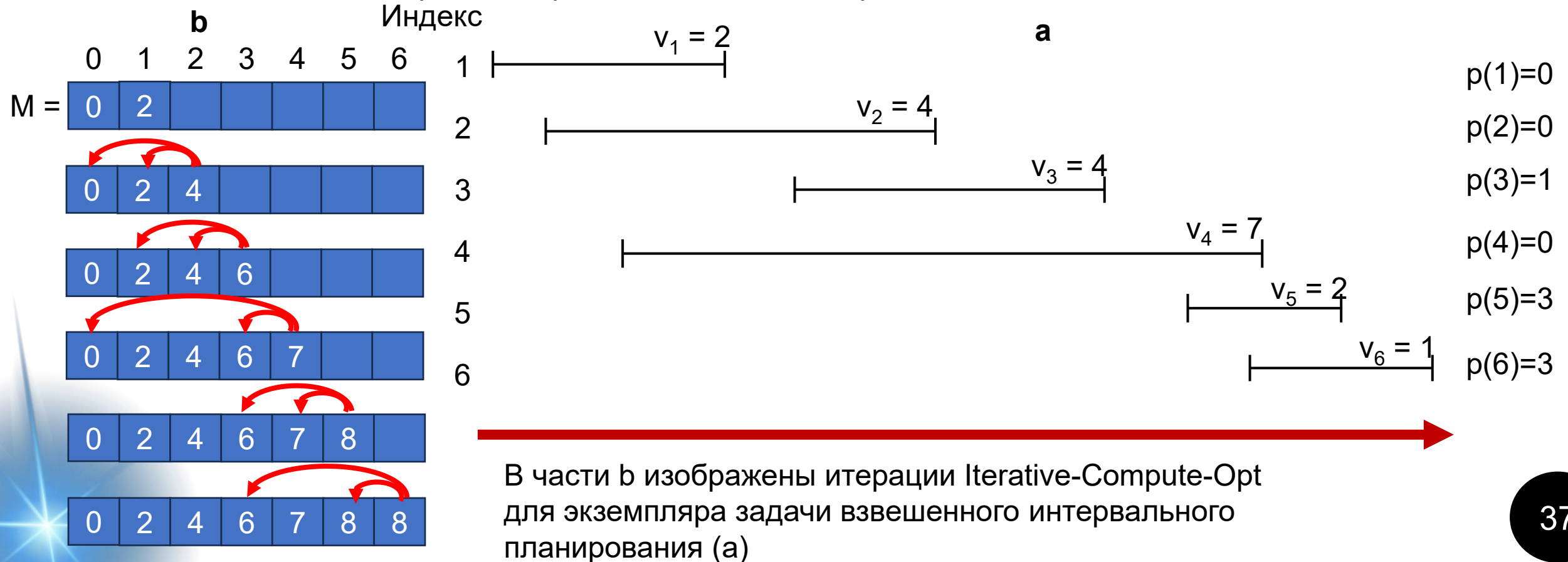
For $j = 1, 2, \dots, n$

$M[j] = \max(v_j + M[p(j)], M[j - 1])$

Конец цикла

Анализ алгоритма

Следуя точной аналогии с доказательством (16.16), можно доказать посредством индукции по j , что этот алгоритм записывает $\text{OPT}(j)$ в элемент массива $M[j]$; (16.14) предоставляет шаг индукции. Кроме того, как и ранее, Find-Solution можно передать заполненный массив M для получения оптимального решения помимо значения. Наконец, время выполнения Iterative-Compute-Opt очевидно равно $O(n)$, так как алгоритм явно выполняется в течение n итераций и проводит постоянное время в каждой из них.



Основная схема динамического программирования

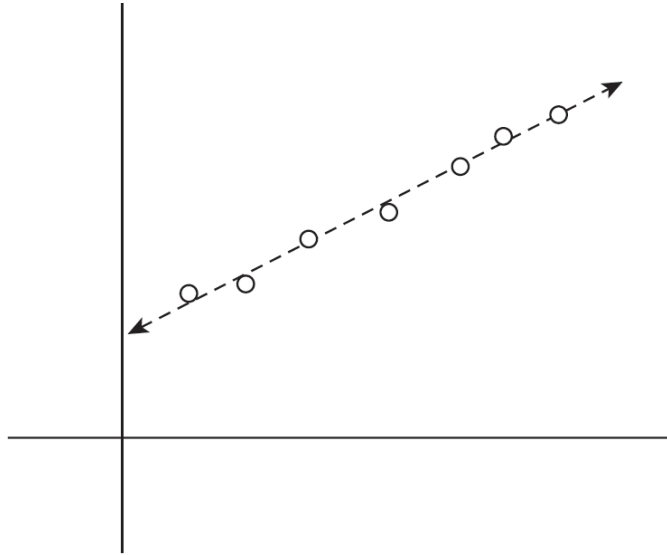
Чтобы взяться за разработку алгоритма, основанного на методе динамического программирования, необходимо иметь набор подзадач, производных от исходной задачи, который обладает некоторыми базовыми свойствами.

- (i) Количество подзадач ограничено полиномиальной зависимостью.
- (ii) Решение исходной задачи легко вычисляется по решениям подзадач. (Например, исходная задача может быть одной из подзадач.)
- (iii) Существует естественное упорядочение подзадач от «меньшей» к «большей» в совокупности с легко вычисляемой рекуррентностью (как в (16.14) и (16.15)), которая позволяет определить решение подзадачи по решениям некоторого количества меньших подзадач.

Связь между подзадачами и рекуррентными отношениями, в которой трудно отделить первопричину от следствия, является одной из тонких особенностей, лежащих в основе динамического программирования. Никогда нельзя быть уверенным в том, что набор подзадач будет полезным, пока не будет найдено рекуррентное отношение, связывающее подзадачи воедино; но о рекуррентном отношении трудно думать в отсутствие «меньших» подзадач, с которыми оно работает.

Сегментированные наименьшие квадраты: многовариантный выбор

При работе с научными и статистическими данными, нанесенными на плоскость, аналитики часто пытаются найти «линию наилучшего соответствия» для этих данных



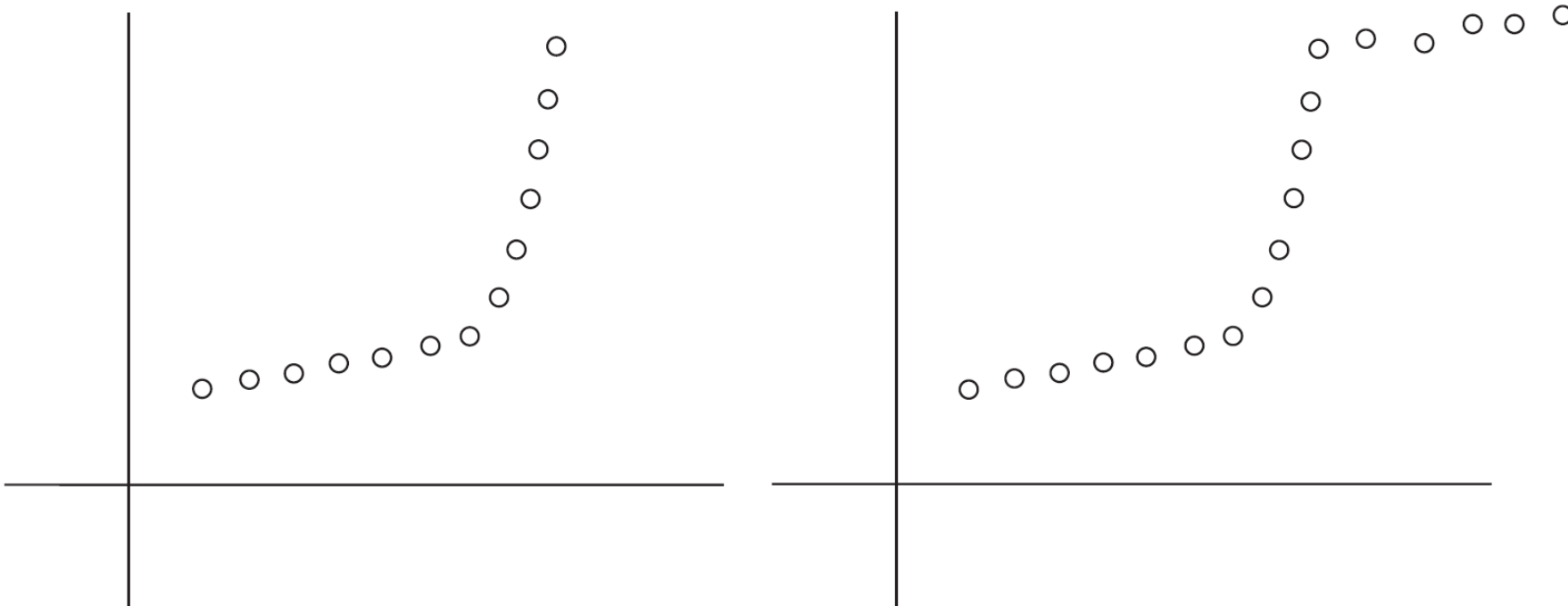
Эта основополагающая задача из области статистики и вычислительной математики. Допустим, данные представляют собой множество P из n точек на плоскости, обозначаемых $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$; предполагается, что $x_1 < x_2 < \dots < x_n$. Для линии L , определяемой уравнением $y = ax + b$, погрешностью L относительно P называется сумма возведенных в квадрат «расстояний» от точек до P :

$$Error(L, P) = \sum_{i=1}^n (y_i - ax_i - b)^2.$$

Естественной целью в такой ситуации будет нахождение линии с минимальной погрешностью. Линия минимальной погрешности определяется формулой $y = ax + b$, где

$$a = \frac{n \sum_i x_i y_i - (\sum_i x_i)(\sum_i y_i)}{n \sum_i x_i^2 - (\sum_i x_i)^2} \text{ и } b = \frac{\sum_i y_i - a \sum_i x_i}{n}.$$

Задача



Как формализовать такие задачи?

1. Ищем набор линий, обеспечивающий минимальную погрешность. Но такая формулировка задачи никуда не годится, потому что у нее имеется тривиальное решение: при сколь угодно большом наборе линий можно добиться идеального соответствия, соединив линиями каждую пару последовательных точек в P .
2. С противоположной стороны можно жестко «запрограммировать» число 2 в задаче — искать лучшее соответствие не более чем с двумя линиями. Но и такая формулировка противоречит здравому смыслу: изначально не было никакой априорной информации о том, что точки лежат приблизительно на двух линиях.

Формулировка задачи

Имеется множество точек $P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, в котором $x_1 < x_2 < \dots < x_n$. Точка (x_i, y_i) будет обозначаться p_i .

- Необходимо разбить P на некоторое количество сегментов, представляющих непрерывный набор координат x ; $\{p_i, p_{i+1}, \dots, p_{j-1}, p_j\}$ для некоторых индексов $i \leq j$.
- Затем для каждого сегмента S в разбиении P вычисляется линия, минимизирующая погрешность в отношении точек S .

Штраф разбиения $Cost = C \times (\text{количество сегментов}) + \sum \text{ошибка_сегмента}$:

- (i) Количество сегментов, на которые разбивается P , умноженное на фиксированный множитель $C > 0$.
- (ii) Для каждого сегмента — значение погрешности для оптимальной линии через этот сегмент.

Нашей целью в сегментированной задаче наименьших квадратов является нахождение разбиения с минимальным штрафом. Минимизация этой характеристики отражает компромиссы, упоминавшиеся ранее. При поиске решения могут рассматриваться разбиения на любое количество сегментов; с увеличением количества сегментов уменьшаются слагаемые штрафа из части (ii) определения, но увеличивается слагаемое в части (i). (Множитель C предоставляется со входными данными; настраивая C , можно увеличить или уменьшить штраф за использование дополнительных линий.)

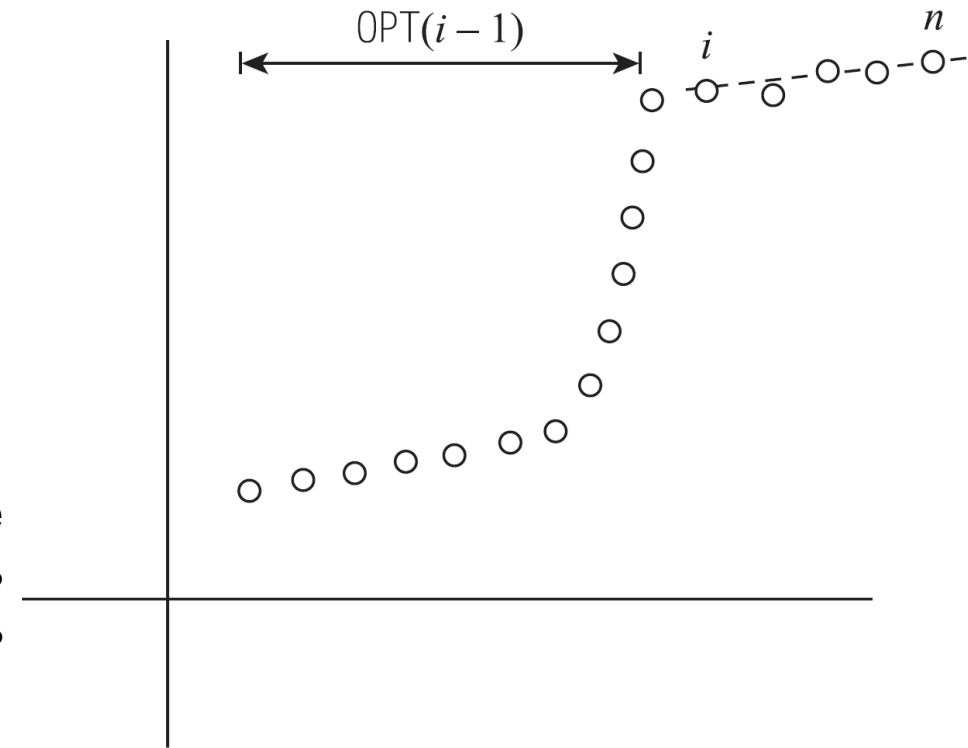
Возможен ли эффективный поиск оптимального разбиения?



Разработка алгоритма

Возможное решение: подбираем один сегмент для точек $p_i, p_{i+1}, \dots, p_n$, после чего зная последний сегмент, мы можем исключить эти точки из рассмотрения и рекурсивно решить задачу для оставшихся точек $p_1, \dots, p_{i-1}$.

Предположим, $OPT(i)$ обозначает оптимальное решение для точек $p_1, \dots, p_i$, а $e_{i,j}$ — минимальную погрешность для любой линии в отношении $p_i, p_{i+1}, \dots, p_j$. (Запись $OPT(0) = 0$ используется как граничный случай.)



Утверждение 16.19. Если последний сегмент оптимального решения состоит из точек $p_i, \dots, p_n$, то значение оптимального решения равно $OPT(n) = e_{i,n} + C + OPT(i - 1)$.

Используя то же наблюдение для подзадачи, состоящей из точек $p_1, \dots, p_j$, мы видим, что для получения $OPT(j)$ необходимо найти лучший способ получения завершающего сегмента $p_i, \dots, p_j$ — с оплатой погрешности и слагаемого C для этого сегмента — в сочетании с оптимальным решением $OPT(i - 1)$ для остальных точек. Другими словами, мы обосновали следующее рекуррентное отношение:

Утверждение 16.20. Для подзадачи с точками $p_1, \dots, p_j$

$$OPT(j) = \min_{1 \leq i \leq j} (e_{i,j} + C + OPT(i - 1)),$$

а сегмент $p_i, \dots, p_j$ используется в оптимальном решении подзадачи в том и только в том случае, если минимум достигается при использовании индекса i .

Разработка алгоритма

Построить решение $OPT(i)$ в порядке возрастания i .

Segmented-Least-Squares(n)

Массив M $[0 \dots n]$

Присвоить $M[0] = 0$

Для всех пар $i \leq j$

 Вычислить наименьшую квадратичную погрешность $e_{i,j}$ для сегмента $p_i, \dots, p_j$

Конец цикла

Для $j = 1, 2, \dots, n$

 Использовать рекуррентное отношение (16.20) для вычисления $M[j]$

Конец цикла

Вернуть $M[n]$

Как и в алгоритме взвешенного интервального планирования, оптимальное разбиение вычисляется обратным отслеживанием по M .

Find-Segments(j)

Если $j = 0$

 Ничего не выводить

Иначе

 Найти значение i , минимизирующее $e_{i,j} + C + M[i - 1]$

 Вывести сегмент $\{p_i, \dots, p_j\}$ и результат $\text{Find-Segments}(i - 1)$

Конец условия

Анализ алгоритма

Время выполнения Segmented-Least-Squares.

Сначала необходимо вычислить значения всех погрешностей наименьших квадратов $e_{i,j}$. Чтобы учесть время выполнения, расходуемое на эти вычисления, заметим, что существуют $O(n^2)$ пар (i, j) , для которых эти вычисления необходимы; для каждой пары (i, j) можно использовать формулу, приведенную в начале этого раздела, для вычисления $e_{i,j}$ за время $O(n)$.

Следовательно, общее время выполнения для вычисления всех значений $e_{i,j}$ равно **$O(n^3)$** , *на самом деле можно быстрее за $O(n^2)$, но это дз.*

Алгоритм состоит из n итераций для значений $j = 1, \dots, n$. Для каждого значения j необходимо определить минимум в рекуррентном отношении (16.20) для заполнения элемента массива $M[j]$; это занимает время $O(n)$ для каждого j , что в сумме дает **$O(n^2)$** . Итак, после определения всех значений $e_{i,j}$ время выполнения равно $O(n^2)$.



Задача о сумме подмножеств и задача о рюкзаке: добавление переменной

Условие: Имеется одна машина, способная обрабатывать задания, и набор заявок $\{1, 2, \dots, n\}$. Ресурсы могут обрабатываться только в период времени от 0 до W (для некоторого числа W). Каждая заявка представляет собой задание, для обработки которого требуется время w_i .

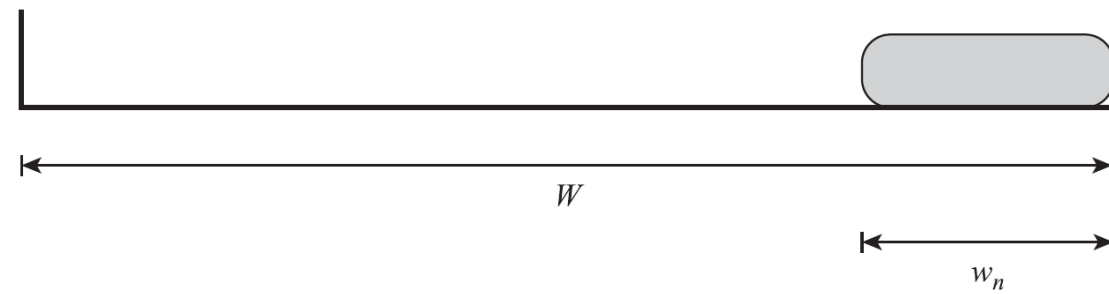
Если нашей целью является обработка заданий, при которой обеспечивается максимальная занятость машины до «граничного времени» W , какие задания следует выбрать?

В более формальном виде: имеются n элементов $\{1, \dots, n\}$, каждому из которых присвоен неотрицательный вес w_i (для $i = 1, \dots, n$). Также задана граница W . Требуется выбрать подмножество S элементов, для которого $\sum_{i \in S} w_i \leq W$, чтобы этого ограничения значение $\sum_{i \in S} w_i$ было как можно больше. Эта задача называется задачей о сумме подмножеств.

Целью в этой общей задачи является нахождение подмножества с максимальным суммарным значением, при котором общий вес не превышает W .

Один из естественных жадных методов основан на сортировке элементов по убыванию веса, а затем выборе элементов в этом порядке, пока общий вес остается меньше W . Но если значение W кратно 2, и имеются три элемента с весами $\{W/2 + 1, W/2, W/2\}$, этот жадный алгоритм не обеспечит оптимального решения.

Разработка алгоритма



Неудачная попытка

Стратегия с рассмотрением подзадач, в которых задействованы только первые i заявок.

Лучшее возможное решение, использующее подмножество заявок $\{1, \dots, i\}$, будет обозначаться $\text{OPT}(i)$

- Если $n \notin O$, то $\text{OPT}(n) = \text{OPT}(n - 1)$.
- Если $n \in O$, то не следует, что какую-то другую заявку нужно отклонить, — а лишь то, что для подмножества принимаемых заявок $S \subseteq \{1, \dots, n - 1\}$ остается меньше свободного веса: вес w_n используется для принятой заявки n , так что для множества S остальных принимаемых заявок остается только вес $W - w_n$.

Улучшенное решение

Для определения значения $\text{OPT}(n)$ необходимо знать не только значение $\text{OPT}(n - 1)$, но и лучшее решение, которое может быть получено с использованием подмножества первых $n - 1$ элементов и общего доступного веса $W - w_n$. Предположим, W — целое число, а все запросы $i = 1, \dots, n$ имеют целые веса w_i .

$$\forall i = 0, 1, \dots, n \quad \forall 0 \leq w \leq W \quad \text{OPT}(i, w) = \max_S \sum_{j \in S} w_j$$

где максимум определяется по подмножествам $S \subseteq \{1, \dots, i\}$, удовлетворяющим условию $\sum_{j \in S} w_j \leq w$. $\text{OPT}(n, W)$ — значение, которое мы ищем в конечном итоге. Как и прежде, пусть O обозначает оптимальное решение исходной задачи.

- Если $n \notin O$, то $\text{OPT}(n, W) = \text{OPT}(n - 1, W)$, так как элемент n можно просто игнорировать.
- Если $n \in O$, то $\text{OPT}(n, W) = w_n + \text{OPT}(n - 1, W - w_n)$, так как теперь ищется вариант оптимального использования оставшейся емкости $W - w_n$ между элементами $1, 2, \dots, n - 1$.

Разработка алгоритма

Когда n -й элемент слишком велик, то есть $W < w_n$, должно выполняться условие $OPT(n, W) = OPT(n - 1, W)$. В противном случае мы получаем оптимальное решение, допускающее все n заявок, выбирая лучший из этих двух вариантов. Применяя аналогичные рассуждения для подзадачи с элементами $\{1, \dots, i\}$ и максимальным допустимым весом w , получаем следующее рекуррентное отношение:

Утверждение 16.21. Если $w < w_i$, то $OPT(i, w) = OPT(i - 1, w)$. В противном случае $OPT(i, w) = \max(OPT(i - 1, w), w_i + OPT(i - 1, w - w_i))$.

Как и прежде, мы хотим создать алгоритм, который строит таблицу всех значений $OPT(i, w)$, вычисляя каждое из них не более одного раза.

Subset-Sum(n, W)

Массив $M[0 \dots n, 0 \dots W]$

Инициализировать $M[0, w] = 0$ для всех $w = 0, 1, \dots, W$

For $i = 1, 2, \dots, n$

For $w = 0, \dots, W$

 Использовать рекуррентное отношение (16.21) для вычисления $M[i, w]$

Конец For

Конец For

Вернуть $M[n, W]$

При помощи (16.21) можно немедленно доказать посредством индукции, что возвращаемое значение $M[n, W]$ является значением оптимального решения для заявок $1, \dots, n$ и доступного веса W .

Анализ алгоритма

n	0													
	0													
	0													
	0													
i	0													
$i-1$	0													
	0													
	0													
	0													
2	0													
1	0													
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		0	1	2	$w - w_i$			w					W	

В качестве примера выполнения этого алгоритма рассмотрим экземпляр с предельным весом $W = 6$ и $n = 3$ элементами с размерами $w_1 = w_2 = 2$ и $w_3 = 3$. Оптимальное значение $\text{OPT}(3, 6) = 5$ достигается при использовании третьего элемента и одного из первых двух элементов.

Размер рюкзака $W = 6$, элементы $w_1 = 2, w_2 = 2, w_3 = 3$

3							
2							
1							
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

Исходные значения

3							
2							
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

Заполнение значений для $i = 1$

3							
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

Заполнение значений для $i = 2$

3	0	0	2	3	4	5	5
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

Заполнение значений для $i = 3$

Двумерная таблица значений OPT. Крайний левый столбец и нижняя строка всегда содержат 0. Значение $\text{OPT}(i, w)$ вычисляется по двум другим элементам — $\text{OPT}(i - 1, w)$ и $\text{OPT}(i - 1, w - w_i)$ в соответствии со стрелками

Анализ алгоритма

Следующий вопрос — время выполнения алгоритма. Как и прежде в задаче взвешенного интервального планирования, мы строим таблицу решений M и вычисляем каждое из значений $M[i, w]$ за время $O(1)$ с использованием предыдущих значений. Следовательно, время выполнения пропорционально количеству элементов в таблице.

Утверждение 16.22. Алгоритм $\text{Subset-Sum}(n, W)$ правильно вычисляет оптимальное значение для задачи и выполняется за время $O(nW)$.

Обратите внимание: этот метод менее эффективен, чем динамическая программа для задачи взвешенного интервального планирования. В самом деле, его время выполнения не является полиномиальной функцией n ; это полиномиальная функция n и W — наибольшего целого, задействованного в определении задачи. Такие алгоритмы называются псевдополиномиальными. Псевдополиномиальные алгоритмы могут обладать неплохой эффективностью при достаточно малых числах $\{w_i\}$ во входных данных; с увеличением этих чисел их практическая применимость снижается.

Чтобы получить оптимальное множество элементов S , можно выполнить обратное отслеживание по массиву M по аналогии с тем, как это делалось в предыдущих разделах:

Утверждение 16.23. Для таблицы M оптимальных значений подзадач оптимальное множество S может быть найдено за время $O(n)$.

Расширение: задача о рюкзаке

Задача о рюкзаке немного сложнее задачи планирования, которая рассматривалась нами ранее. Возьмем ситуацию, в которой каждому элементу i поставлен в соответствие неотрицательный вес w_i , как и прежде, и отдельное значение v_i . Цель — найти подмножество S с максимальным значением $\sum_{i \in S} v_i$, в котором общий вес множества не превышает W : $\sum_{i \in S} w_i \leq W$.

Алгоритм динамического программирования несложно расширить до этой более общей задачи. Аналогичное множество подзадач $\text{OPT}(i, w)$ используется для представления значения оптимального решения с использованием подмножества элементов $\{1, \dots, i\}$ и максимального доступного веса w . Рассмотрим оптимальное решение O и определим два случая в зависимости от того, выполняется ли условие $n \in O$:

- Если $n \notin O$, то $\text{OPT}(n, W) = \text{OPT}(n - 1, W)$.
- Если $n \in O$, то $\text{OPT}(n, W) = v_n + \text{OPT}(n - 1, W - w_n)$.

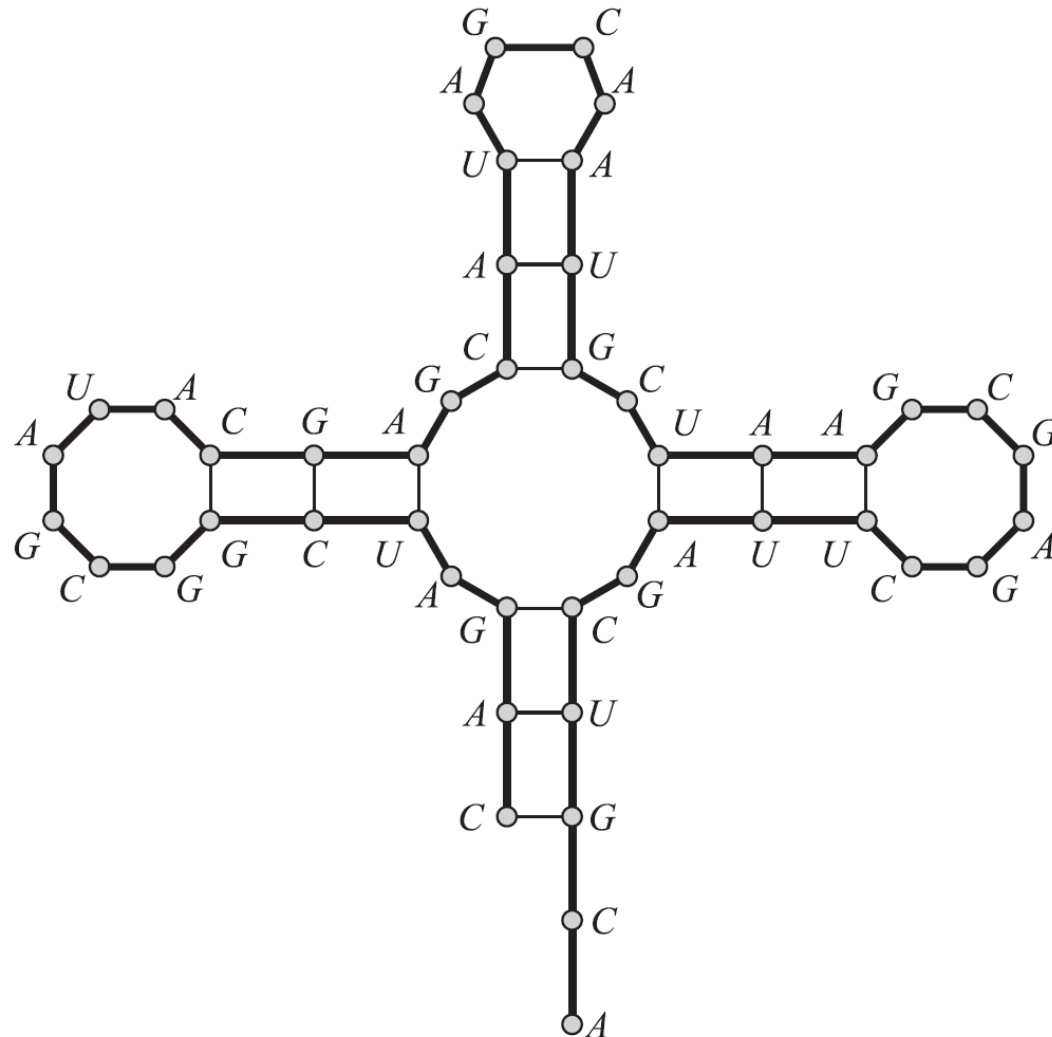
Применение этой аргументации к подзадачам приводит к следующей аналогии с (16.21).

Утверждение 16.24. Если $w < w_i$, то $\text{OPT}(i, w) = \text{OPT}(i - 1, w)$. В противном случае $\text{OPT}(i, w) = \max(\text{OPT}(i - 1, w), v_i + \text{OPT}(i - 1, w - w_i))$.

При помощи этого рекуррентного отношения можно записать полностью аналогичный алгоритм динамического программирования, из чего вытекает следующий факт:

Утверждение 16.25. Задача о рюкзаке решается за время $O(nW)$.

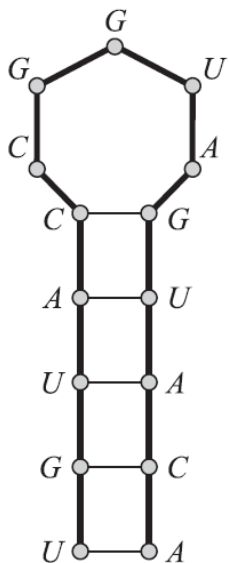
Вторичная структура РНК: динамическое программирование по интервалам



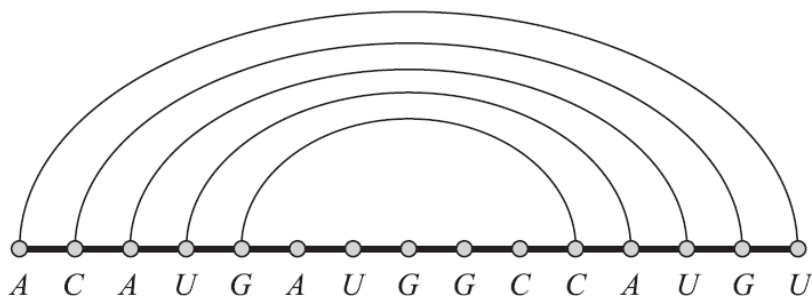
Уотсон и Крик установили, что двухцепочечная ДНК «связывается» комплементарным спариванием оснований. Каждая цепь ДНК может рассматриваться как цепочка оснований, каждое из которых выбирается из множества $\{A, C, G, T\}$. Пары образуются как основаниями A и T, так и основаниями C и G. Одноцепочечные молекулы РНК являются ключевыми компонентами многих процессов, происходящих в клетках. У РНК не существует «второй цепи» для связывания, поэтому РНК обычно образует пары оснований сама с собой; это приводит к образованию интересных форм. Совокупности пар (и полученная форма), образованные молекулой РНК в этом процессе, называются вторичной структурой; понимание вторичной структуры необходимо для понимания поведения молекулы. Для наших целей одноцепочечная молекула РНК может рассматриваться как последовательность из n символов (оснований) алфавита $\{A, C, G, U\}$.

Пусть $V = b_1 b_2 \dots b_n$ — одноцепочечная молекула РНК, в которой все $b_i \in \{A, C, G, U\}$.

Задача



a



b

В первом приближении вторичная структура может моделироваться следующим образом. Как обычно, основание A образует пары с U, а основание C образует пары с G; также требуется, чтобы каждое основание могло образовать пару с не более чем одним другим основанием, иначе говоря, множество пар оснований образует паросочетание. Также вторичные структуры (снова в первом приближении) не образуют узлов, что ниже формально определяется как своего рода условие отсутствия пересечений.

А если более конкретно, вторичная структура V представляет собой множество пар $S = \{(i, j)\}$, где $i, j \in \{1, 2, \dots, n\}$, удовлетворяющих следующим условиям:

- (i) (Отсутствие крутых поворотов) Концы каждой пары в S разделяются минимум четырьмя промежуточными основаниями; иначе говоря, если $(i, j) \in S$, то $i < j - 4$.
- (ii) Любая пара в S состоит из элементов $\{A, U\}$ или $\{C, G\}$ (в произвольном порядке).
- (iii) S является паросочетанием; никакое основание не входит более чем в одну пару.
- (iv) (Отсутствие пересечений) Если (i, j) и (k, l) — две пары в S , то невозможна ситуация $i < k < j < l$.

Разработка и анализ алгоритма

Динамическое программирование: первая попытка

Мы говорим, что $OPT(j)$ — максимальное количество пар оснований во вторичной структуре $b_1, b_2, \dots, b_j$. Согласно приведенному выше запрету на резкие повороты, мы знаем, что $OPT(j) = 0$ для $j \leq 5$; также известно, что $OPT(n)$ — искомое решение.

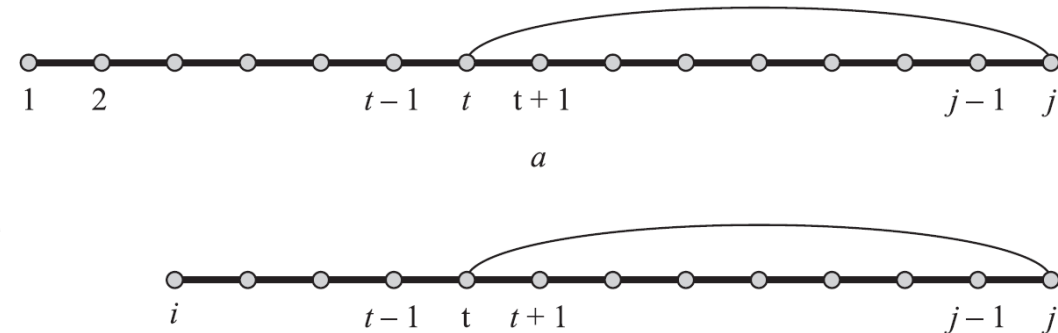
Как записать рекуррентное отношение, выражающее $OPT(j)$ в контексте решений меньших подзадач?

Здесь можно пройти часть пути: в оптимальной вторичной структуре $b_1, b_2, \dots, b_j$ возможно одно из двух:

- j не участвует в паре; или
- j участвует в паре с t для некоторого $t < j - 4$.

Включение пары (i, j) порождает две независимые подзадачи

В первом случае необходимо обратиться к решению для $OPT(j - 1)$. Второй случай изображен на **рисунке, а**; из-за запрета на пересечения мы знаем, что не может существовать пары, один конец которой находится между 1 и $t - 1$, а другой — между $t + 1$ и $j - 1$. Таким образом, мы фактически выделили две новые подзадачи: для оснований $b_1 b_2 \dots b_{t-1}$ и для оснований $b_{t+1} \dots b_{j-1}$.



Первая решается как $OPT(t - 1)$, а вторая в наш список подзадач не входит, потому что она не начинается с b_1 . Это соображение наводит на мысль о том, что в задачу следует добавить переменную. Нужно иметь возможность работать с подзадачами, не начинающимися с b_1 ; другими словами, необходимо иметь возможность рассматривать подзадачи для $b_i b_{i+1} \dots b_j$ при любых $i \leq j$.

Разработка и анализ алгоритма

Динамическое программирование по интервалам

После принятия этого решения приведенные выше рассуждения ведут прямо к успешной рекурсии. Пусть $OPT(i, j)$ — максимальное количество пар оснований во вторичной структуре $b_i b_{i+1} \dots b_j$. Запрет на резкие повороты позволяет инициализировать $OPT(i, j) = 0$ для всех $i \geq j - 4$.

(Для удобства записи мы также разрешим ссылки $OPT(i, j)$ даже при $i > j$; в этом случае значение равно 0).

Теперь в оптимальной вторичной структуре $b_i b_{i+1} \dots b_j$ существуют те же альтернативы, что и прежде:

- j не участвует в паре; или
- j находится в паре с t для некоторого $t < j - 4$.

В первом случае имеем $OPT(i, j) = OPT(i, j - 1)$. Во втором случае, порождаются две подзадачи $OPT(i, t - 1)$ и $OPT(t + 1, j - 1)$; как было показано, эти две задачи изолируются друг от друга условием отсутствия пересечений. Следовательно, мы только что обосновали следующее рекуррентное отношение:

Утверждение 16.26. $OPT(i, j) = \max(OPT(i, j - 1), \max_t(1 + OPT(i, t - 1) + OPT(t + 1, j - 1)))$, где $\max$ определяется по t , для которых b_t и b_j образуют допустимую пару оснований (с учетом условий (i) и (ii) из определения вторичной структуры).

Теперь нужно убедиться в правильном понимании порядка построения решений подзадач. Из (16.26) видно, что решения подзадач всегда вызываются для более коротких интервалов: тех, для которых $k = j - i$ меньше. Следовательно, схема будет работать без каких-либо проблем, если решения строятся в порядке возрастания длин интервалов.

Разработка и анализ алгоритма

Инициализировать $OPT(i,j) = 0$ для всех $i \geq j-4$

For $k = 5, 6, \dots, n - 1$

For $i = 1, 2, \dots, n - k$

 Присвоить $j = i + k$

 Вычислить $OPT(i,j)$ с использованием рекуррентного отношения из (16.26)

Конец For

Конец For

Вернуть $OPT(1,n)$

Найти границу для времени выполнения несложно: существуют $O(n^2)$ подзадач, которые необходимо решить, а вычисление рекуррентного отношения в (16.26) занимает время $O(n)$ для каждой задачи. Следовательно, время выполнения будет равно $O(n^3)$.

РНК последовательность ACCGGUAGU

4	0	0	0	
3	0	0		
2	0			
$i=1$				
	$j=6$	7	8	9

Исходные значения

4	0	0	0	0
3	0	0	1	
2	0	0		
$i=1$	1			
	$j=6$	7	8	9

Заполнение значений для $k = 5$

4	0	0	0	0
3	0	0	1	1
2	0	0	1	
$i=1$	1	1		
	$j=6$	7	8	9

Заполнение значений для $k = 6$

4	0	0	0	0
3	0	0	1	1
2	0	0	1	1
$i=1$	1	1	1	
	$j=6$	7	8	9

Заполнение значений для $k = 7$

4	0	0	0	0
3	0	0	1	1
2	0	0	1	1
$i=1$	1	1	1	2
	$j=6$	7	8	9

Заполнение значений для $k = 8$

Выравнивание последовательностей

Задача

Словари в Интернете становятся все более удобными: часто бывает проще открыть сетевой словарь по закладке, чем снимать бумажный словарь с полки. К тому же многие электронные словари предоставляют возможности, отсутствующие в бумажных словарях: если вы ищете определение и введете слово, отсутствующее в словаре (например, **ocurrance**), словарь поинтересуется: «Возможно, вы имели в виду **occurrence**?» Как он это делает? Он действительно знает, что происходит у вас в голове?

В начале 1970-х годов молекулярные биологи Нидлман и Вунш предложили определение сходства, которое практически в неизменном виде стало стандартным определением, используемым в наши дни.

Допустим, имеются две строки X и Y ; строка X содержит последовательность символов $x_1, x_2, \dots, x_m$, а строка Y — последовательность символов $y_1, y_2, \dots, y_n$. Рассмотрим множества $\{1, 2, \dots, m\}$ и $\{1, 2, \dots, n\}$, представляющие разные позиции в строках X и Y , и рассмотрим паросочетание этих множеств. Паросочетание M этих двух множеств называется выравниванием при отсутствии «пересекающихся» пар: если $(i, j), (i', j') \in M$ и $i < i'$, то $j < j'$. На интуитивном уровне выравнивание предоставляет способ параллельного расположения двух строк: оно сообщает, какие пары позиций будут находиться друг напротив друга. Например,

stop

-tops

соответствует выравниванию $\{(2, 1), (3, 2), (4, 3)\}$.

Выравнивание последовательностей

Задача

Наше определение сходства будет основано на нахождении оптимального выравнивания между X и Y по следующему критерию. Предположим, M — заданное выравнивание между X и Y .

- *Во-первых*, должен существовать параметр $\delta > 0$, определяющий штраф за разрыв. Для каждой позиции X или Y , не имеющей пары в M (то есть разрыва), вводится штраф δ .
- *Во-вторых*, для каждой пары букв p, q в нашем алфавите существует штраф за несоответствие α_{pq} за совмещение p с q . Таким образом, для всех $(i, j) \in M$ совмещение x_i с y_j приводит к выплате соответствующего штрафа за несоответствие $\alpha_{x_i y_j}$. В общем случае предполагается, что $\alpha_{pp} = 0$ для любого символа p , то есть совмещение символа со своей копией не приводит к штрафу за несоответствие — хотя это предположение не является обязательным для каких-либо дальнейших рассуждений.
- Стоимость выравнивания M вычисляется как сумма штрафов за разрывы и несоответствия. Требуется найти выравнивание с минимальной стоимостью.

В литературе по биологии процесс минимизации стоимости часто называется **выравниванием последовательностей**. Величины δ и $\{\alpha_{pq}\}$ представляют собой внешние параметры, которые должны передаваться программе для выравнивания последовательностей; и действительно, значительная часть работы связана с выбором значений этих параметров. При разработке алгоритма выравнивания последовательностей мы будем рассматривать их как заданные извне. Возвращаясь к первому примеру, обратите внимание на то, как эти параметры определяют, какое из выравниваний *occurrence* и *occurrence* следует предпочесть: первый вариант строго лучше других в том и только в том случае, если $\delta + \alpha_{ae} < 3\delta$.

Разработка алгоритма

Для оценки сходства между строками X и Y имеется конкретное числовое определение: это минимальная стоимость выравнивания между X и Y . Для решения этой задачи можно попытаться применить динамическое программирование, руководствуясь следующей базовой дихотомией.

- В оптимальном выравнивании M либо $(m, n) \in M$, либо $(m, n) \notin M$. (То есть два последних символа двух строк либо сопоставляются друг с другом, либо нет.)

Утверждение 16.27. Пусть M — произвольное выравнивание X и Y . Если $(m, n) \notin M$, то либо m -я позиция X , либо n -я позиция Y не имеют сочетания в M .

Доказательство. Действуя от обратного, предположим, что $(m, n) \notin M$ и существуют числа $i < m$ и $j < n$, для которых $(m, j) \in M$ и $(i, n) \in M$. Но это противоречит нашему определению выравнивания: имеем $(i, n), (m, j) \in M$ с $i < m$, но $n > j$, поэтому пары (i, n) и (m, j) пересекаются. ■

Существует эквивалентный вариант записи (16.27) с тремя альтернативными возможностями, приводящий напрямую к формулировке рекуррентного отношения.



Разработка алгоритма

Утверждение 16.28. В оптимальном выравнивании M истинно хотя бы одно из следующих трех условий:

- (i) $(m, n) \in M$; или
- (ii) m -я позиция X не имеет сочетания; или
- (iii) n -я позиция Y не имеет сочетания.

Теперь пусть $OPT(i, j)$ обозначает минимальную стоимость выравнивания между $x_1x_2 \cdots x_i$ и $y_1y_2 \cdots y_j$. Если выполняется условие (i) из (16.28), мы платим и выравниваем $x_1x_2 \cdots x_{m-1}$ с $y_1y_2 \cdots y_{n-1}$ настолько хорошо, насколько это возможно; получаем $OPT(m, n) = \alpha_{x_my_n} + OPT(m-1, n-1)$. Если выполняется условие (ii), мы платим штраф за разрыв δ , потому что m -я позиция X не имеет сочетания, и выравниваем $x_1x_2 \cdots x_{m-1}$ с $y_1y_2 \cdots y_{n-1}$ настолько хорошо, насколько это возможно. В этом случае получаем $OPT(m, n) = \delta + OPT(m-1, n)$. Аналогичным образом для случая (iii) получаем $OPT(m, n) = \delta + OPT(m, n-1)$.

Используя аналогичные рассуждения для подзадачи нахождения выравнивания с минимальной стоимостью для $x_1x_2 \cdots x_i$ и $y_1y_2 \cdots y_j$, приходим к следующему факту:

Утверждение 16.29. Стоимости минимального выравнивания удовлетворяют следующему рекуррентному отношению для $i \geq 1$ и $j \geq 1$:

$$OPT(i, j) = \min[\alpha_{x_iy_j} + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1)].$$

Кроме того, (i, j) принадлежит оптимальному выравниванию M для этой подзадачи в том и только в том случае, если минимум достигается на первом из этих значений.

Мы добрались до точки, в которой алгоритм динамического программирования стал ясен: мы строим значения $OPT(i, j)$, используя рекуррентное отношение в (16.29). Существуют только $O(mn)$ подзадач, и $OPT(m, n)$ дает искомое значение.

Разработка алгоритма

Теперь определим алгоритм для вычисления значения оптимального выравнивания. Для выполнения инициализации заметим, что $OPT(i, 0) = OPT(0, i) = i\delta$ для всех i , так как совместить i -буквенное слово с 0-буквенным словом возможно только с использованием i разрывов.

Alignment(X, Y)

Массив $A[0\dots m, 0\dots n]$

Инициализировать $A[i, 0] = i\delta$ для всех i

Инициализировать $A[0, j] = j\delta$ для всех j

For $j = 1, \dots, n$

For $i = 1, \dots, m$

 Использовать рекуррентное отношение из (16.29) для вычисления $A[i, j]$

Конец For

Конец For

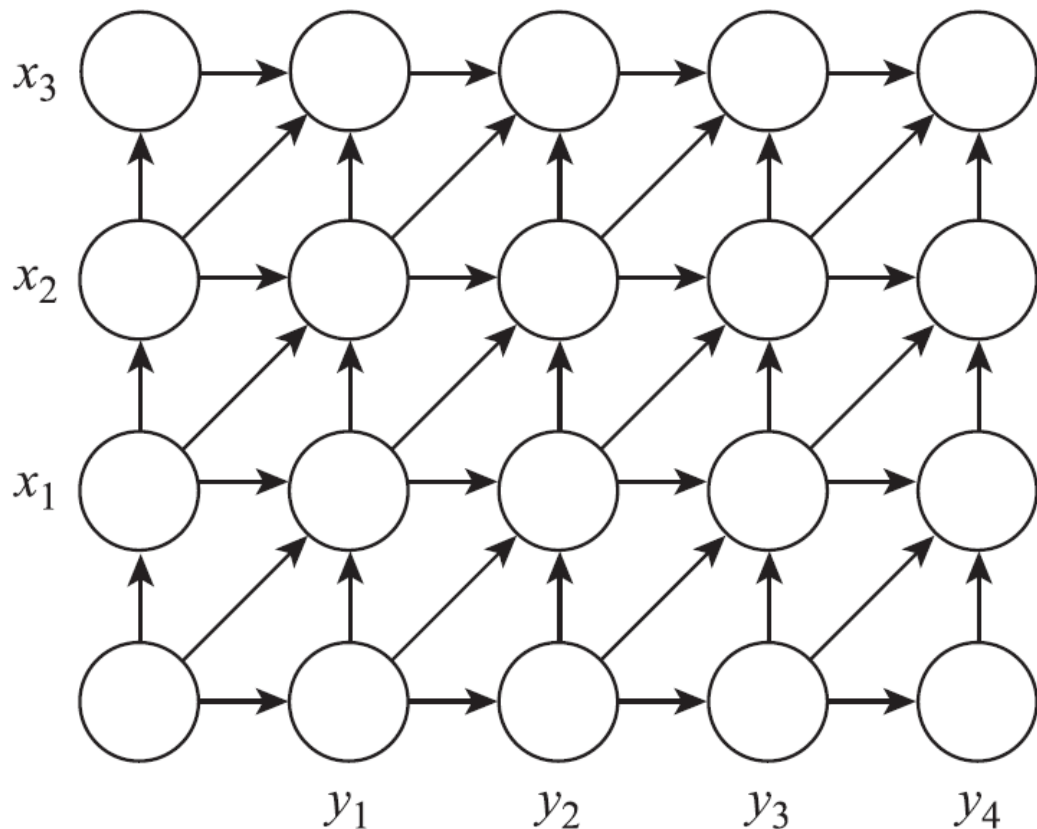
Вернуть $A[m, n]$

Как и в случае с предыдущими алгоритмами динамического программирования, построение самого выравнивания осуществляется обратным отслеживанием по массиву A с использованием второй части факта (16.29).

Анализ алгоритма

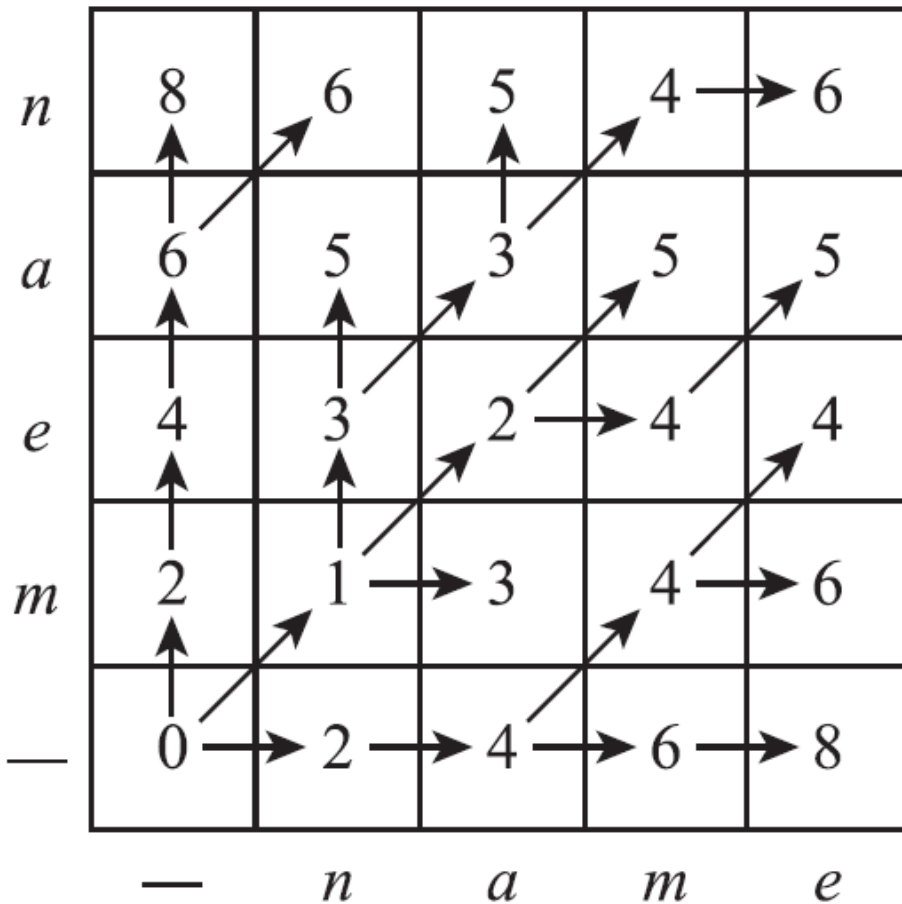
Правильность алгоритма следует непосредственно из (16.29). Время выполнения равно $O(mn)$, так как массив A содержит только $O(mn)$ элементов и в худшем случае на каждый тратится постоянное время.

У алгоритма выравнивания последовательностей существует элегантное визуальное представление. Допустим, имеется двумерный решетчатый граф GXY с размерами $m \times n$; строки помечены символами X , столбцы помечены символами Y , а ребра направлены так, как показано на рисунке



Строки нумеруются от 0 до m , а столбцы от 0 до n ; узел на пересечении i -й строки и j -го столбца обозначается (i, j) . Ребрам GXY назначаются стоимости: стоимость каждого горизонтального и вертикального ребра равна δ , а стоимость диагонального ребра из $(i - 1, j - 1)$ в (i, j) равна $\alpha_{x_i y_j}$.

Анализ алгоритма



Значения OPT для задачи
выравнивания слов mean и name

Теперь смысл этой схемы становится понятным: рекуррентное отношение для $OPT(i, j)$ из (16.29) в точности соответствует рекуррентному отношению для пути минимальной стоимости от $(0, 0)$ до (i, j) в GXY. Следовательно, мы можем показать

Утверждение 16.30. Пусть $f(i, j)$ — стоимость минимального пути из $(0, 0)$ в (i, j) в GXY. Тогда для всех i, j выполняется условие $f(i, j) = OPT(i, j)$.

Доказательство. Утверждение легко доказывается индукцией по $i + j$. Если $i + j = 0$, значит, $i = j = 0$, и тогда $f(i, j) = OPT(i, j) = 0$.

Теперь рассмотрим произвольные значения i и j и предположим, что утверждение истинно для всех пар (i', j') с $i' + j' < i + j$. Последнее ребро кратчайшего пути к (i, j) проходит либо из $(i - 1, j - 1)$, либо из $(i - 1, j)$, либо из $(i, j - 1)$. Следовательно, имеем

$$\begin{aligned} f(i, j) &= \min[\alpha_{x_i y_j} + f(i - 1, j - 1), \delta + f(i - 1, j), \delta + f(i, j - 1)] \\ &= \min[\alpha_{x_i y_j} + OPT(i - 1, j - 1), \delta + OPT(i - 1, j), \delta + OPT(i, j - 1)] \\ &= OPT(i, j), \end{aligned}$$

Переход от первой строки ко второй осуществляется по индукционной гипотезе, а переход от второй к третьей — по (16.29).